A project report on

AYULEAF VISION: AI-BASED SYSTEM FOR AYURVEDIC PLANT DISEASE DETECTION AND ANALYSIS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

S R KOUSALYA (22BCE5225)

AKAASH K (22BCE5218)

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

APRIL, 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the project entitled "AYULEAF VISION: AI-BASED SYSTEM FOR AYURVEDIC PLANT DISEASE DETECTION AND ANALYSIS" submitted by S R KOUSALYA (22BCE5225), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai, is a record of bonafide work carried out by me under the supervision of Dr. SUSEELA S.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: 08/04/2026

S.R. Kousalya

Signature of the Candidate

S R KOUSALYA

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"AYULEAF VISION: AI-BASED SYSTEM FOR AYURVEDIC PLANT DISEASE DETECTION AND ANALYSIS"** is prepared and submitted by **S R KOUSALYA (22BCE5225)** to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Suseela S

Date: 8/4/2026

Signature of the Examiner

Name: Dr. V.R. BALASAKTHI SWATHI

Date: 9/4/2026

Signature of the Examiner

Name: Mr. Mohammed Hussain

Date: 9/4/26 Head of Architect Engg

Approved by the Head of Department,

Computer Science and Engineering

Name: **Dr. Prassanna J**

Date: 8/4/26

VELLORE INSTITUTE OF TECHNOLOGY * Chennai-600127 * SCHOOL OF COMPUTER SCIENCE & ENGINEERING (SCOPE)

ABSTRACT

The Ayurvedic herbs like Neem, Tulsi, Turmeric and Aloe Vera play a major therapeutic role in the traditional medicine systems that are practised in South Asia. Nonetheless, precise diagnosis of diseases in the said plants has been a thorn in the flesh of farmers and herbalists especially in the field where limited resources exist to engage the services of the expert in diagnosing the disease. The traditional means of detecting diseases is the manual visual approach.

In this project, the proposed pipeline is an end-to-end intelligent disease detection and diagnostic model of plant disease that uses the techniques of deep learning. The system uses a convolutional neural network architecture MobileNetV2, which is a lightweight model that is pretrained on the ImageNet dataset and is fine-tuned on a subset of images of leaves, namely 13,460 such images in 18 disease classes and 4 species of ayurvedic plants. Before model inference, the images of the leaves are enhanced with the help of Contrast Limited Adaptive Histogram Equalization (CLAHE) in the LAB colour space, which significantly increases the visualisation of the disease-related information in the conditions of different light levels.

The trained model attained a test accuracy of 90.13 percent and weighted F1 score of 89.87 percent which showed to be a strong model of classification of various disease groups. In addition to classification, the system includes the lesion segmentation using HSV to localise and quantify the disease pathology at the pixel level and Gradient-weighted Class Activation Mapping (Grad-CAM) to explain the model decisions visually and a leaf colour statistics module evaluating the greenness and quality of the colours. A field parameter processing is done in an environmental analysis component processing the field parameters of temperature, humidity, and soil moisture.

All the diagnostic outputs are combined into the system to form a final report in a well-formatted file that includes the predicted disease type, confidence level, level of disease severity, leaf health value, image quality evaluation, and treatment recommendation which is specific to the plant. The report can be exported in the form of excel file and this can be deployed to the field. The combination of a classification, segmentation, explainability, and an analysis of the environment into one pipeline makes this work a significant advance towards the accessibility of technology-based precision agriculture of ayurvedic plants cultivation.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Suseela S, our project guide, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for her constant guidance, continual encouragement, understanding; more than all, she taught me patience in my endeavor. My association with her has not only been confined to academics, but it is a great opportunity on my part to work with an intellectual and expert in the field of Wireless Multimedia Sensor Networks.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan, our Honorable Chancellor, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G.V. Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai, Dr. K. Sathiyanarayanan, Director, Chennai Campus and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Dr. Suganya G, Dr. Sweetlin Hemalatha C, Associate Deans, Associate Deans, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

I express my sincere gratitude to Dr. Prassanna J, Head of the Department, B.Tech. Computer Science and Engineering, and the Project Coordinators for their valuable support and encouragement to take up and complete this thesis.

My sincere thanks to all the faculties and staff at Vellore Institute of Technology, Chennai, who helped me acquire the requisite knowledge. I would like to thank my parents for their unwavering support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: 08/04/2026

S. R. Kousalya

S R KOUSALYA

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS

Acronym	Full Form
AI	Artificial Intelligence
CNN	Convolutional Neural Network
CLAHE	Contrast Limited Adaptive Histogram Equalization
DL	Deep Learning
Grad-CAM	Gradient-weighted Class Activation Mapping
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value
LAB	Lightness, A-channel, B-channel (CIE Colour Space)
ML	Machine Learning
MobileNetV2	Mobile Network Version 2
RGB	Red, Green, Blue
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
TL	Transfer Learning
VGG	Visual Geometry Group
ResNet	Residual Neural Network
EfficientNet	Efficient Neural Network
F1	F1-Score (Harmonic Mean of Precision and Recall)
API	Application Programming Interface
CSV	Comma Separated Values

Chapter 1

INTRODUCTION

1.1 BACKGROUND OF THE STUDY

The medicinal plant cultivation tradition in India is one of the richest in the world and Ayurveda is a form of healthcare system that is centuries old and based its treatment interventions on botanical sources virtually completely. They can deduce four species among the enormous ayurvedic plant pharmacopoeia, which include Neem (Azadirachta indica), Tulsi (Ocimum tenuiflorum) Turmeric (Curcuma longa) and Aloe Vera (Aloe barbadensis) owing to their extensive cultivation, recorded medicinal effect, and economic exploitation in India.

Neem also has a high application in its antibacterial, antifungal, and anti-inflammatory effects. Tulsi is being adored because of respiratory, immunological and antimicrobial effects. As a bioactive compound, curcumin is found in turmeric and is used in the treatment of anti-inflammatory, anti-oxidant and wound healing. Aloe Vera is extensively used in the skin care, digestive and against burns.

The budding process is, however, by no means a simple one. Similar to any other agricultural commodity, they are prone to various bacteria, fungi, and environmental ailments, which appear as the observable ways of change in the morphology, colour, texture, and structure of the leaf. Leaf blotch, bacterial infection, defoliator damage, anthracnose, aloe rust and sunburn are diseases that may lead to significant losses in the health of plants, yield and quality of medicinal compound extracted. The diagnosis of these diseases at early and accurate stages is of high importance to facilitate the prompt intervention, minimize lost revenue through loss of crop and retain the therapeutic contents of the harvested plant materials.

Manual visual inspection by trained agronomists or botanists is still the primary method usually used in the traditional diagnosis of plant diseases. Although it is an invaluable expertise, it is literally of a finite scale: the conditions of the field are different, the agreement of the diverse observers is not one hundred percent, and human specialists may not be constantly available in rural areas of cultivation.

What is more, early symptoms are characterized by similar visuals with various diseases and are therefore unseen with the naked eye, which is most likely to produce an error. Such shortcomings have been a source of inspiration to the scientific community to come up with automated technology-based solutions to diagnosis of plant diseases.

Due to the rapid development of deep learning processes in the last decade, the automated image-based disease detection becomes a viable proposal and accurate to a significant extent. Convolutional neural nets (CNNs) are particularly adapted to leaf disease classification problems as they can be trained to learn gradient features (i.e. edge and colour gradient) in a hierarchical manner, up to disease lesion patterns, across raw pixel data themselves with no human-centric feature extraction. This has enabled the faster application of CNN-based plant disease detection algorithms in agricultural research due to a combination of large datasets of labeled plant images, inexpensive access to on-the-fly computing with a graphics card, and high-level neural networks like TensorFlow, and PyTorch.

Transfer learning especially has been disruptive in this field. Through the use of pretrained weights in the CNN, specifically ImageNet, a large-scale general image dataset, instead of training the optimally on a relatively small plant disease dataset, scholars have repeatedly shown that it is feasible to reach high levels of accuracy with limited domain-specific data and reduced training time. VGG16, ResNet50, Inception V3, EfficientNet, and MobileNetV2 are all architectures that have all been applied with a significant degree of success to plant pathology classification tasks.

The specific network architecture of the current project is MobileNetV2 that is a lightweight CNN built on mobile and embedded applications. Its convolution structure can be separated depthwise, reducing the number of model parameters and cost of computation by an enormous margin relative to more dense architectures, and gives it competitive classification performance. This is what makes it especially applicable to the possible on-device application in field conditions, where the available amount of computational resources is limited.

1.2 PROBLEM STATEMENT

Although the research on the application of deep learning to detect plant diseases is increasing, in the parameter of ayurvedics and its medicinal plants, there are still some practical limitations that are not tackled. To start with, the existing datasets and trained models are mostly concerned with food crops, i.e. rice, wheat, maize, and tomato. Ayurvedic plants Datasets of ayurvedic plants, including the entire range of diseases that affect Neem, Tulsi, Turmeric, and Aloe Vera, are relatively few and are frequently lacking annotations to help identify a fine-grained disease classification.

Second, the majority of classification systems that exist today are black box: they produce a predicted label describing the classification, and do not give any information about the regions of the leaf that motivated the prediction at the spatial level. The end result of such obscurity is that it destroys the confidence of the user especially when it comes to farmers and the healthcare professionals who require the ability to visualize the diagnosis. Online explainability tools like Grad-CAM that create class-discriminative localisation maps that signify the parts on the image that have the greatest contribution to the model prediction are crucial but often not often in deployed plant disease classification systems.

Third, classification is not enough to have pragmatic care of illnesses. Being aware of the type of disease is not enough but it is required and the farmer must know the intensity of the attack of the plant in order to tell to what extent the treatment response will be urgency and intensity dependent. This vital dimension of information is given by pixel-level quality of scans that is based on lesion segmentation.

Fourth, empirical leaf images are not of optimal quality, they may be in bad light, have unstable contrast, be blurred due to motion, or even be obscured by soil or other participants. Processing pipelines that are robust in amplitude of improving image quality before classification are hence good but not always included in the present systems.

This project bridges these gaps with an integrated pipeline that integrates the concepts of transfer learning classification on MobileNetV2, image improvement on CLAHE, lesion segmentation on HSV, Grad-CAM explainability, and environmental analysis into a system constructed as one system that fully relies on ayurvedic plants.

1.3 OBJECTIVES

The main goals of this project will be:

- To develop and optimize a MobileNetV2 based Transfer learning model that can classify the leaf diseases of 18 classes for four ayurvedic plant species with a test accuracy of more than 85.
- Hypothesis: H 0: It is hypothesised that while using CLAHE-based image enhancement in LAB colour space does not enhance the quality and discriminability of leaf images before model inference, it improves the quality and discriminability of leaf images derived from model inference.
- It will be desired that it creates an HSV-based lesion segmentation module that will isolate diseased regions on the pixel level and group them into Low, Medium and High assessments of disease severity.
- Besides the above, the main objective of the research is to incorporate Grad-CAM visualisation in generating spatial heatmaps to see where the leaf regions have the most impact on the model prediction and therefore offer interpretable and reliable diagnostic.
- To add a leaf colour statistics module together with an environmental analysis component that will place the diagnosis in context relative to physiological and field contexts.
- To produce organised, exportable diagnosis report and this will contain all the diagnostic output and give specific treatment advice based on the plant.

1.4 SCOPE OF THE PROJECT

The project scope comprises the following:

The paper confronts the disease detection and severity analysis pipeline of four ayurvedic medicinal plants, Neem, Tulsi, Turmeric, and Aloe Vera, in the context of 18 disease and healthy groups based on a set of 13,460 leaf images. The scope includes data preparation, model training, image enhancement, lesion segmentation,

explainability, environmental analysis, and report generation, all developed in Python with support of TensorFlow and OpenCV with the usage of the Google Colab service and the help of the GPU.

The project does not involve real-time video stream processing, multi-leaf batch processing of images individually, combination with embedded hardware systems, and clinical validation in agronomic field tests. These dimensions are natural extensions in the future work but they are out of this scope.

1.5 ORGANISATION OF THE REPORT

The Report was organised in the following way:

The rest of this report will be structured in the following way. Chapter 2 conducts a literature review of the current state of knowledge regarding deep learning-based plant disease detectors, transfer learning, image improvement, and explainability conceptualization, indicating the gaps in the literature on the subject that are filled by the present project. Chapter 3 explains the system design and methodology and outlines all the elements of the diagnostic pipeline. Chapter 4 includes the code descriptions of all modules that are in consideration. The experimental results are discussed in chapter 5 and contain model performance measures, visualisations and diagnostic output correlates. Chapter 6 is the final part of the report and philosophizes about limitations, future work directions. References and appendices follow.

Chapter 2

LITERATURE REVIEW

2.1 INTRODUCTION

The Automated detection of plant diseases by analyzing images has been of such research interest since the beginning of the 2010s due to the development of convolutional neural networks and the release of large annotated datasets of plant images. The chapter of this paper provides a literature review in five topic areas, namely deep learning-based plant disease recognition algorithms, transfer learning algorithms, agricultural image-enhancement algorithms, explainability algorithms like Grad-CAM, and severity quantification algorithms. The second aspect that the chapter ends with is the specification of the research gaps that served to guide the structure of this research.

2.2 PLANT DISEASE DETECTION USING DEEP LEARNING

The original work of Mohanty et al. (2016) was the first example of a deep convolutional neural network that was able to categorize 26 diseases among 14 species of crops using leaf images under controlled settings with more than 99 percent accuracy. They used the PlantVillage dataset on their work that soon emerged as the standard of the future studies on plant pathology. Nevertheless, further analysis revealed that models that were trained with PlantVillage images (collected on uniform backgrounds with regulated illumination) rendered weakly to images of the fields with natural backgrounds and can change with regards to illumination.

V. Pratima (2018) performed a study of the performance of various CNN models such as AlexNet, VGGNet, and GoogLeNet on the PlantVillage dataset by affirming that deep CNNs had significantly high relative performance to more traditional machine learning methods such as support residences machines on custom-made features. The investigation revealed that the depth and width of architectural buildings did contribute so much to the accuracy of classification.

S. Padgilwar et al. (2025) tested the application of a deep CNN to detect the image based identification, and evaluated a variety of pretrained networks, demonstrating that the fine-tuning method was more effective than the feature extraction method with

smaller domain-specific outcomes. This paper demonstrated the practical usefulness of transfer learning in comparison with training in scratch training in places of plant pathology.

Transplanting the application of lightweight architecture to a mobile deployment support, A. Mohanadevi, and others (2026) assessed the MobileNetV1, MobileNetV2, and ShuffleNet architectures applied to limiting datasets of medicinal plants . Their findings substantiated that MobileNetV2 was the highest accuracy to computation trade-off with lightweight models, thus it was chosen as the candidate of the use in mobile plants health monitoring. They are accurate on a five disease dataset at 88.9 percent, which is generally similar to the 90.13 percent of their current system on a much more difficult 18 class one.

R. Kumar et al. (2023) created a leaf diseases severity estimation system based on deep CNNs and found that, with the addition of image augmentation approaches (rotation, flipping, and zoom) the effect of overfitting was minimised and the process was enhanced as disease detection was improved with unknown test images. This observation took a place in the augmentation process that is undertaken within the current project.

2.3 TRANSFER LEARNING IN AGRICULTURAL APPLICATIONS

Transfer learning has become the paradigm of choice on training and classifying agricultural images tasks where the training data is usually labelled compared with the technical complexity of the prediction task. ImageNet-trained networks have pretrained weight matrices that contain high-level visual features, such as, edges, textures, object parts, etc, which are well transferred to the problem of plant pathology though the difference in domain.

The article by S. S. Srambickal et al. (2016) was one of the first to use transfer learning based on a general-purpose CNN to plant disease detection with a pretrained network based on non-agricultural imagery and fine-tuning it on leaf disease. They found that transfer learning achieved a much better accuracy as compared to training on the same amount of domain data when learning on a blank (training from scratch), especially on underrepresented disease classes.

A. Iqbal and L. Damahe (2025) carried out a systematic study of nine deep CNN and compared them with each other using the plantvillage data, including those of VGG16, VGG19, ResNet, InceptionV3, and DenseNet. The best results with inceptionV3 was the highest accuracy of 96.46 though at a great cost in terms of increasing the computation needs. The authors observed that in case of edge deployment, models with accuracy more than 85% take the form of an acceptably good accuracy-efficiency trade-off.

J. Lande et al. (2025) have studied MobileNetV2 imparticularly in plant disease classification and expressed the identical test accuracy as heaviest architectures and inferred over five times as quickly. This efficiency and accuracy resulted in the MobileNetV2 was especially suitable to be applied in the context of mobile deployment which is the vision of the current project.

As it was shown in the study by A. R. Revathi et al. (2025), the stagnation of deeper convolutional base of a pretrained MobileNet and the subsequent training of the specific classification head, i.e. matching the best models in this project, resulted in convergence at a rate that is almost three times faster than fine-tuning all the layers, at a closely negligible cost of about an additional percentage-point in accuracy.

2.4 IMAGE ENHANCEMENT TECHNIQUES

Raw leaf images that are recorded in the field environment are often characterised by uneven light distribution, low contrast between healthy and diseased areas, and colour distortions which hinder correct disease detection by human observers as well as classifiers. The use of preprocessing to enhance image has thus been looked upon as a based on the enhancement of below stream classification performance.

One of the oldest and most commonly used contrast enhancements techniques, which is still popular, is histogram equalisation (HE) and is based on the redistribution of pixel values in the entire range of dynamic values. Although it works well on globally low-contrast images, at the localized bright or dark areas, HE is prone to artificially enhancing noise causing unnatural artefacts in a texture-rich image of leaves.

Adaptive histogram equalisation (AHE) is a solution of this weakness where equalisation of local image tiles is carried out to conserve local contrast, but does not cause global intensity distortion (as opposed to non-local methods). Nevertheless,

AHE increases noise in the nearly constant areas. CLAHE (Contrast Limited Adaptive Histogram Equalization) extends AHE by adding a clip limit that limits the amount of contrast enhancement factor per tile and essentially hissing down noise amplification process which does not diminish the local contrast enhancing.

It used CLAHE preprocessing on rice disease images and then classified them with CNN reported a 3.2% increase in the classification accuracy of these with preprocessing rather than use of unenhanced input. The researchers specifically identified that, CLAHE related to the enhancement of the detection of the uncovered lesions with low performances in relation to the leaf background which was healthy.

The color space used by the current project is the LAB colour space instead of working on the three RGB colour channels. Working with the L (lightness) channel and no modifications to A and B channels (identifying colour) would mean that the enhancement will be done to luminance only without altering the chromatic structure that carries the disease specific colour changes. It is an effective method that has been confirmed in similar studies of colour-critical tasks of imaging agricultural colours.

The automated detection of plant diseases through image analysis has attracted substantial research interest since the early 2010s, catalysed by advances in convolutional neural networks and the publication of large annotated plant image datasets. This chapter surveys the relevant literature across five areas: deep learning methods for plant disease detection, transfer learning approaches, image enhancement techniques for agricultural imaging, explainability methods including Grad-CAM, and severity quantification strategies. The chapter concludes by identifying the specific research gaps that motivated the design of this project.

2.5 EXPLAINABILITY AND GRAD-CAM

The opacity of deep neural network predictions has been a persistent concern in high-stakes agricultural and medical applications, where users need to understand why the model arrived at a particular decision in order to assess its reliability and act upon it with confidence. Class Activation Mapping (CAM) methods address this concern by generating spatial visualisations of the image regions that most influenced the model output.

They introduced the original Class Activation Mapping technique, which leverages the global average pooling layer of a CNN to produce class-discriminative localisation maps without requiring any architectural changes or re-training. While effective, the original CAM was limited to architectures containing a global average pooling layer directly before the classifier.

They proposed Gradient-weighted Class Activation Mapping (Grad-CAM), which generalized CAM to any CNN architecture by using the gradients of the class score with respect to the feature maps of the last convolutional layer as importance weights. Grad-CAM produced visually interpretable heatmaps highlighting which spatial regions of the input image activated the most strongly for the predicted class, without requiring architectural modifications.

In plant disease detection, Grad-CAM has been used to verify that models are attending to the biologically meaningful leaf regions — lesions, discoloured areas, structural damage — rather than incidental background features. Mohanty et al. verified via CAM analysis that their PlantVillage-trained CNN focused on the lesions themselves rather than the uniform background, which raised the question of whether this attention would generalise to field images with natural backgrounds.

The present project implements Grad-CAM on the out_relu layer of the MobileNetV2 architecture to generate per-prediction heatmaps overlaid on the enhanced leaf image. This provides users with an interpretable visual confirmation that the model prediction is grounded in the actual disease lesion rather than background noise.

2.6 SEVERITY ANALYSIS METHODS

It is not enough to know the type of disease in order to begin doing this; by discovering the percentage of diseased plant tissue farmers are able to tune the degree and irrepressibleness of treatment. The form of the severity analysis of the plant disease system is a separation of diseased areas into pixels and calculation of the ratio of infected to total leaf pixels.

The early severity predictors were based on colour thresholding within RGB space and this method makes use of diseased pixels by their nonconformity to the desired healthy green of a healthy leaf tissue. The operation of RGB thresholding is however sensitive to changes in illumination and the operation cannot effectively differentiate between disease symptoms and natural leaf colour gradation.

It has been demonstrated that HSV (Hue, Saturation, Value) colour space is found to be stronger in lesion segmentation in diverse illumination conditions since the hue is comparatively unchanged to the brightness of the light, and saturation reproduces the purity of colour regardless of brightness. In a systematic review of colour-based lesion segmentation algorithms, that HSV-based thresholding always performed better than RGB-based thresholding of field-acquired images.

The morphological operations of erosion, dilation, opening and closing are typically used as post-processing measures following HSV thresholding resulting in the elimination of small spurious lesion detection and closing of gaps between larger lesion areas to enhance the precision of the severity estimate. This is what is done in the present project which morphologically opens and closes after the HSV segmentation.

2.7 RESEARCH GAPS

The analysis of the existing body of literature suggests various gaps that overall contribute to the justification of the designing of the current project. First, it lacks deep learning systems dedicated to ayurvedic pharmaceutical products and particularly authenticated. Most of the systems published are devoted to food crops and Neem, Tulsi, Turmeric and Aloe Vera are underutilized regardless of the importance of agricultural and pharmaceutical role.

Secondly, the vast majority of published systems deal with the classification part only and fail to combine severity quantification, explainability, or the environmental context analysis into the pipeline of solutions of a single diagnostic process. Farmers need not only a disease name but also an estimate of the severity and recommendations to take action to make informed treatment choices.

Third, the effects on a classification task of image enhancement preprocessing - namely CLAHE - have not been examined experimentally as applied to ayurvedic plant disease data, so it is not clear that the improvement of classification performances observed with food crop data would also apply to this field.

Fourth, the input of environmental parameters (temperature, humidity, soil moisture) as contextual inputs and image-acquired diagnostics have not been much of a concern in published systems of plant diseases although there are well-established agronomic relationships between environmental conditions and disease prevalence and severity.

The current project brings together all four of these gaps in one and integrated system

Chapter 3

DESIGN AND METHODOLOGY

3.1 INTRODUCTION

This chapter presents the architectural structure and research methodology which supports the ayurvedic plant disease identification and diagnostic system. The system operates through a series of sequential stages which require each stage to use the previous stage's results as its input while providing a unique diagnostic analysis. The stages of implementation include (1) preparing the dataset (2) training the model through transfer learning (3) enhancing images (4) classifying diseases (5) removing backgrounds and segmenting lesions (6) demonstrating explanations through Grad-CAM (7) assessing disease severity (8) extracting color statistics (9) examining environmental conditions and (10) creating reports.

3.2 SYSTEM ARCHITECTURE OVERVIEW

The detected plant disease and severity analysis system proposed is an intelligent pipeline in multi-stages and sequential processing in an attempt to analyze a raw leaf image by custom analytical modules and generate an overall diagnostic output. Figure 3.1 gives an overall picture of the architecture. The system combines end-to-end systems and methods of deep learning-based classification with computer vision based segmentation, explainability mechanisms, and environmental reasoning, all made available through a web application interface.

Input Layer

A pipeline can be fed on any one of the four target ayurvedic plants, namely, Neem, Tulsi, Turmeric or Aloe Vera. The picture can be obtained with the help of the natural lighting outdoors and in special conditions indoors and implies the submission of the picture by the user on the upload page of the web application. The image capture device is not limited and the system is available in the field of use using any normal smartphone or camera.

Image Preprocessing

Before being classified, all the input images undergo an exclusive preprocessing process. The picture is initially scaled down to 224x224 pixels according to the size needed by the input of the MobileNetV2 structure. This is followed by contrast Limited Adaptive Histogram Equalisation (CIELAB colour space) which is applied to CIE LAB colour space and which works on luminance (L) channel only without altering chromatic A and B channels. This local contrast and visibility enhancement enhances the faint disease feature (e.g. early-stages lesions and discolouration) in the local contrast and visibility of cyanobacteria phyto-plankton and does not distort the colour content, which the classifier uses to identify the disease at hand. To normalise the input distribution, pixel values are eventually scaled to [0 1] by dividing it with 255 in order to infer the model stabilisation.

Data augmentation operations, such as random rotation, horizontal flipping, zoom, width and height shifting, only applied to the training set in the course of training include data augmentation operations to the same preprocessing pipeline, and help in better generalisation and reducing overfitting of the model. The training data are comprised of 13,460 leaves images in 18 disease and healthy classes, processed into a predefined dataset flattening procedure, which rearranges the existing nested directory structure, and transforms it into a flat structure of classes which can be loaded via the Keras data loading pipeline.

MobileNetV2 Classifier

The system has been constructed around the basic component of classification, which is the MobileNetV2, a light-weight convolutional neural network structure trained on the ImageNet dataset. The already trained convolutional base is loaded together with its weights frozen such that the general representations in terms of visual features acquired by large scale image data are not deleted. An expertly classification head is also affixed to the frozen base, containing a Global Average Pooling layer which is able to condense the spatial feature maps into one feature map, followed by a Dense layer of 128 neurons with ReLU activation to carry out non-linear feature transformation, , a Dropout layer of 0.3 to regularise and finally a Dense layer of 18 neurons with Softmax activation to create a probability

To train the model, there is the use of the Adam optimiser with learning rate of 1×10^{-4} and categorical crossentropy loss, EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint callbacks to make sure that the model maximally converges and does not overfit the data. The trained model provides 90.13 with a weighted F1-score of 89.87 with an excellent performance of classification in the entire percentage of diseases in category.

Predicted Disease Class

The classification of the MobileNetV2 classifier is the classification of the disease that will be predicted and a confidence score that indicates how sure the model that will be used makes its prediction. The output of this step is the main point of damage in the pipeline: the class and confidence score are predicted are sent at once to three parallel analytical modules lesion segmentation, Grad-CAM explainability and colour analysis, respectively, all of which provide the diagnosis with a different interpretative lens.

Parallel Analysis Modules

Lesion Segmentation identifies the areas of disease in the leaf on a pixel basis. Through grayscale thresholding and morphologic procedures, the enhanced input image is then first cleared of the background to isolate the leaf and the image background. This is followed by HSV colour space masking which is needed to determine brownish/yellowish pixels which are typical of disease lesions, and this uses lesion masking to form a binary lesion mask that outlines the affected areas of tissue in the leaf.

Grad-CAM (Gradient-weighted Class Activation Mapping) will produce a heatmap of locations that the MobileNetV2 model paid attention to in order to make its prediction. Per-channel importance weights are generated by obtaining gradients of the predicted class score with regard to the feature maps of the final convolutional layer and average-pooling them to get global scores. The heatmap thus created will be visible on the viewable version of the leaf image and this will have a visual confirmation that the model to be used is actually paying attention to biologically relevant areas of disease as opposed to non-biological artefacts.

The Colour Analysis works out the quantitative statistics of leaf colour of the enhanced image in terms of the mean values of the Red, Green, and Blue channels, the mean Hue and Saturation and Value of the representation in HSV, and a Greenness Index of the difference between the mean green channel and the mean of the red and blue channels. The physiological condition of the leaf is measured using these statistics and Good, Moderate, or Poor colour rating of the leaf is determined.

Secondary Analysis Modules

The three components of the secondary analysis take the inception of the outputs of the parallel modules. The Sement of disease severity The severity Analysis involves the calculation of the proportion of the lesion pixels count in the entire pixel count of the image of the HSV lesion mask, with the findings falling into a Low (< 10 percent), Medium (10-30 percent), or High (above 30 percent) category. Image Quality Check is an automatic method that determines the appropriateness of the given input image in the sense of resolution, mean brightness, and Laplacian-based blur score and signals those images which might affect the reliability of the prediction process. The Environmental Analysis uses a rule-based reasoning module to interpret the field parameters that have been given by the user (in the format of temperatures, relative humidity, soil moisture, etc.), to produce agronomic remarks that place the risk of disease in the context of the existing field conditions.

Diagnosis Engine

The results of all the analysis modules are combined in the Diagnosis Engine that the predicted disease type, the confidence level, the severity level, the leaf health status, the colour quality determined, the image quality determined, the environment comments, and the plant-specific treatment recommendation are all in one diagnostic result. The combination of predicted class and the level of severity determines the state of health as Healthy (0), Mildly Affected (1, 2, and 3), Moderately Affected (4, 5, 6, and 7), and Severely Affected (8, 9). The treatment recommendations are developed using a conditional logic function that corresponds a given agronomic recommendation of a pairing of plant species and severity.

Web Application Interface

A web application interface provides the end users with access to the diagnostic pipeline and run three components. The Upload Page gives the user an opportunity to submit a leaf image to be analyzed. Result Dashboard gives a full diagrammatic diagnostic output such as the original image, the enhanced image, lesion mask, Grad-Cam heat map and overlay, severity pie chart and all diagnostic measure in a systematic nine-panelled format. The Report View shows the diagnosis report in readable format where all the fields that have been computed appear.

Output

The system does generate three types of outputs. The 9-panel diagnostic dashboard is used as a full view indicator and hence gives an overall picture of all the pipeline outputs on one screen. The diagnosis report shows the totality of the textual summary of the diagnostic results. Every field of the diagnostic is exported to an organised spreadsheet file to which the predicted class, the confidence, health, the state of the colour, the severity, environmental parameters, the status of the image, and the suggested treatment are all stored in spreadsheet format to be keeped and analysed further.

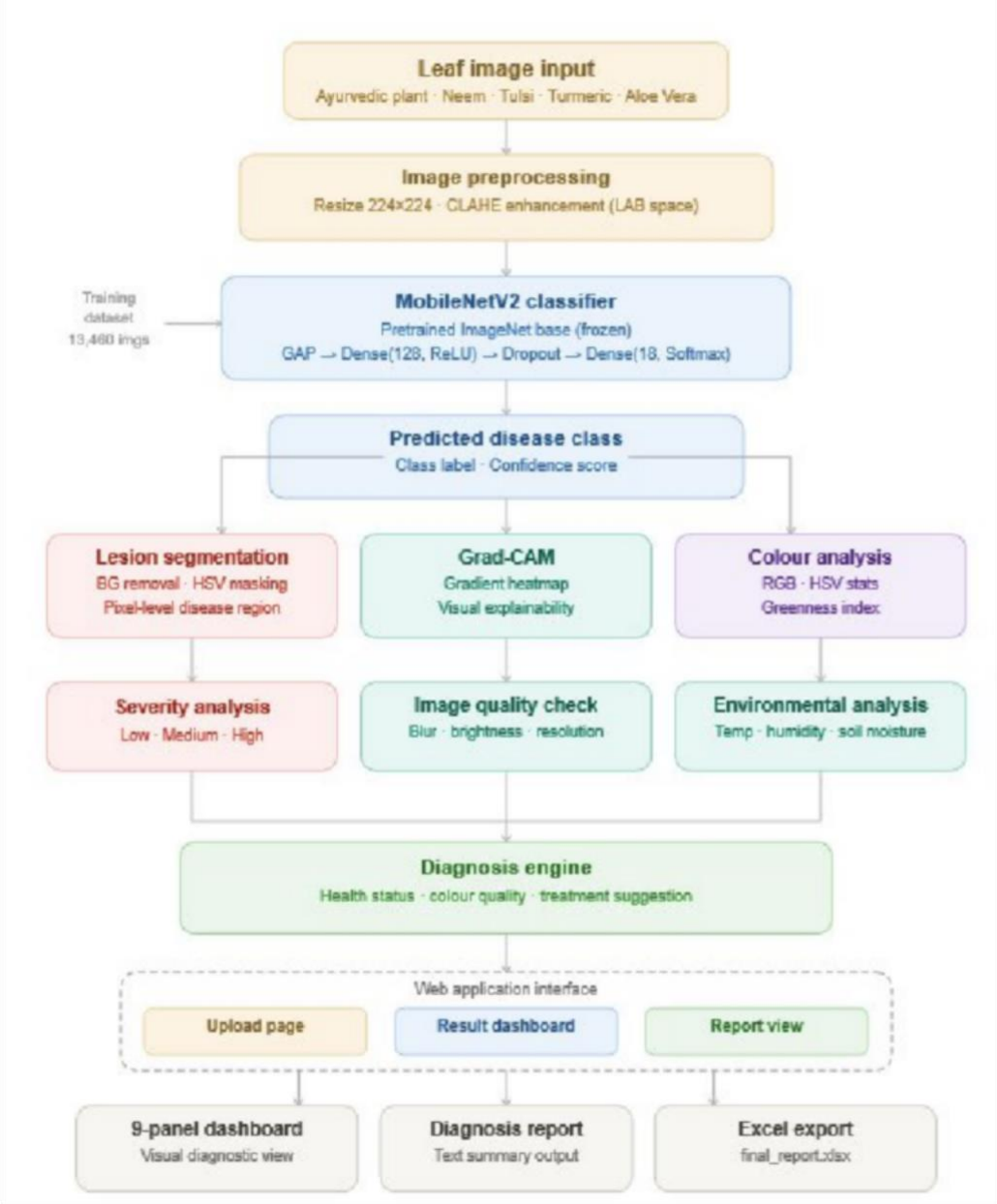

Figure 3.2: System Architecture Diagram

3.3 DATASET DESCRIPTION

The dataset used in this project was compiled from various sources which resulted in a collection of 13460 leaf images divided among 18 classes representing both disease and healthy leaves from four ayurvedic plant species. The original dataset had a nested directory structure where each plant species occupied its own top-level directory while the subdirectories inside that directory contained the different disease conditions. The disease subdirectories contained images which either stored images directly or used further nested subdirectories to organize them based on their source.

Plant	Condition / Disease	No. of Images
Neem	Defoliator	1,559
Neem	Irregular Yellowing	~700
Neem	Leaf Blotch	~600
Neem	Bacterial Infection	~550
Neem	Dieback	~500
Neem	Healthy	~800
Tulsi	Bacterial	~600
Tulsi	Fungal	~550
Tulsi	Pests	~500
Tulsi	Healthy	611
Turmeric	Dry Leaf	203
Turmeric	Leaf Blotch	199
Turmeric	Healthy	~450
Aloe Vera	Leaf Spot	500
Aloe Vera	Anthracnose	~480
Aloe Vera	Aloe Rust	~460
Aloe Vera	Sunburn	~450
Aloe Vera	Healthy	~500

Table 3.1 Shows how the dataset's different classes are distributed.

3.4 DATA PREPROCESSING AND AUGMENTATION

The training was done with a custom flatten dataset() function that produced a flat class directory structure by restructuring the nested directory structure to be acceptable by Keras ImageDataGenerator. The role searched the source directory tree to include image files with extensions (.jpg .jpeg .png .bmp .webp) and then moved them to specific target directories with the label of classes that followed the pattern PlantName__ConditionName.

The 13460 flattened images were split into 3 groups using the splitfolders library that used a constant random seed of 42 to make it reproducible.

This procedure produced 9417 training images and 2686 validation images and 1357 test images. Keras ImageDataGenerator was used to data augment the training set by

applying the augmentation process when loading the data. Table 3.2 has the augmentation parameters.

Augmentation Technique	Parameter Value
Rescaling	1/255 (pixel normalisation)
Rotation Range	20 degrees
Zoom Range	0.2
Width Shift Range	0.1
Height Shift Range	0.1
Horizontal Flip	Enabled

Table 3.2: Data Augmentation Parameters Applied During Training

Keras ImageDataGenerator was used to data augment the training set by applying the augmentation process when loading the data. Table 3.2 has the augmentation parameters.

3.5 MODEL ARCHITECTURE: MOBILENETV2

MobileNetV2 is a lightweight CNN that was presented by Sandler et al. (2018) of Google. The system extends depthwise separable convolutions of MobileNetV1 in two key innovations namely inverted residuals and linear bottlenecks. Standard residual blocks use skip connections to link layers which have numerous channels between the broad layers and the bottleneck layers that use fewer channels.

MobileNetV2 reverses this architecture: the skip connections are made between the bottleneck layers of narrow dimensions, the intermediate layers of broad dimensions of the network. MobileNetV2 is a good options to use in mobile and embedded systems since the architecture reduces multiplication-accumulation processes required to perform a forward pass.

The project loaded MobileNetV2 with ImageNet pretrained weights while excluding the top classification layers (include_top=False). The convolutional base (base_model.trainable = False) that was frozen by the system also permitted only the custom classification head to be trained.

The custom head also had a GlobalAveragePooling2D layer that reshaped the feature maps spatial dimensions into a single feature and a Dense layer with 128 neurons and

the ReLU activation together with a Dropout layer of 0.3 rate in regularisation and a final Dense layer with 18 neurons and softmax activation to give multi-class probability results.

Component	Details
Base Model	MobileNetV2 (ImageNet pretrained)
Input Shape	224 × 224 × 3
Base Model Trainable	No (frozen)
Total Parameters	2,424,274 (9.25 MB)
Trainable Parameters	166,290 (649.57 KB)
Non-trainable Parameters	2,257,984 (8.61 MB)
Custom Head	GAP → Dense(128, ReLU) → Dropout(0.3) → Dense(18, Softmax)
Optimizer	Adam (lr = 1e-4)
Loss Function	Categorical Cross-entropy

Table 3.3: MobileNetV2 Model Parameters Summary

3.6 TRAINING CONFIGURATION

The model was trained using Adam optimiser and categorical crossentropy loss with a learning rate of 1 x 10 -4. A batch size of 16 was used to train the system on 10 maximum epochs. Three callbacks were used to regulate training behaviour:

EarlyStopping checked the loss of validation at a patience of 3 epochs and reloads the best weights at the end at the loss of validation to avoid overfitting.

The system slowed down its learning rate when there was no decrease in validation loss across two consecutive epochs since the system had to perform more precise results in later training phases. The model weights with the highest validation accuracy were stored in ModelCheckpoint to maintain the best model retention during all the further training.

This chapter gives an elaborate account of the architectural design and the methodological approach under which the ayurvedic plant disease detection and diagnosis system was developed.

The system is modelled as a multi-stage pipeline where stage is followed by another stage, where the output of a stage is the input of the next stage and where each stage of the pipeline adds a different layer of analysis to the final diagnosis. These phases include: (1) dataset preparation, (2) model training through the use of transfer learning, (3) image enhancement, (4) disease classification, (5) background removal and lesion segmentation, (6) Grad-CAM explainability, (7) severity analysis, (8) colour statistics extraction, (9) environmental analysis and (10) report generation.

3.7 IMAGE Enhancing with CLAHE

Contrast Limited Adaptive Histogram Equalisation (CLAHE) was a preprocessing tool that was used by operators on input leaf images before they were identified. The improvement process works in the subsequent order

1. The BGR image (loaded by OpenCV) is resized to 224 x 224 pixels The image is converted into the RGB colour space.
2. The RGB image is converted into the CIE LAB colour space.
3. The RGB image that divides luminance information L channel with colour information A and B channels.
4. CLAHE works on the L lightness channel with a clip limit of 3.0 and 8×8 pixels tile grid system The clip limits indicate how much contrast a tile may have which will limit the possibility of excess noise being amplified.
5. The original Channels A and B are left untouched and combined with the improved L channel.
6. The LAB image is transformed back to RGB mode to be displayed and processed.

The LAB colour space gives the fundamental qualifiers to perform CLAHE actions because it divides between luminance amplification and colour credibility that allows the preservation of chromatic disease indicators that the classifier would require to achieve precise diagnosis.

3.8 LESION SEGMENTATION

Lesion segmentation involves two types of steps starting with background removal and incorporating the final step of isolating the lesion areas.

3.8.1 BACKGROUND REMOVAL

The process of background removal begins with the enhanced RGB image which gets converted into grayscale. This process starts with a more efficient conversion of RGB image into grayscale. This is initiated through better RGB image conversion to grayscale. The algorithm commences with improved RGB image processing to grayscale. The algorithm starts by improving RGB image to grayscale conversion. The procedure commences with better conversion of RGB images to grayscale. It starts with the improved RGB image conversion to grayscale. It starts with the improved reconstruction of RGB images into grayscale.

The step starts with improved conversion of RGB images to grayscale. It starts with the improved RGB image conversion to grayscale. The improved RGB image goes to grayscale. The improved RGB picture is equivalent to grayscale. Binary thresholding is then applied with a threshold value of 25, and the result would be a background mask that would outline the area of the leaves on the image. Morphological opening (erosion and dilation) removes minor spurious white areas of the mask and morphological closing (dilation and erosion) replenishes any postholes in the leaf area. An image of a leaf with no background is a masked image derived after the image enhancement mask operating using the bitwise AND drawing.

3.8.2 LESION ISOLATION

The image with background removed is transformed to HSP colour space. Boundaries that define lower bound [10, 30, 30] and upper bound [35, 255, 255] are marked by color range of leaf disease lesions that range with 0 and 180 hues of OpenCV. Any pixel between this range is identified as potential lesion areas. The resulting lesion mask undergoes morphological opening and closing operations which work to remove noise while filling in adjacent lesion spaces.

3.9 GRAD-CAM EXPLAINABILITY

The Grad-CAM sets spatial heatmaps that identify the areas in the image that provide the most contribution to the outcomes of leaf image prediction.

1. A gradient model is created with the help of Keras functional API that allows reaching the results of the last out relu of the convolutional layer and the classification result.

2. The model stores the calculation of the predicted class in a gradient tape of their forward pass of the enhanced input image processing.

3. The research team calculates Grad-CAM class score gradients which they compute based on the feature maps of the final convolutional layer.

4. The gradients on the spatial dimension are averaged globally and a weight vector is obtained that indicates the weight of each channel of each channel of the feature map.

5. Raw Grad-CAM heatmap is a summation of weighted feature map with ReLU activation to remove negative value and is normalized to a range between 0 and 1.

6. The heatmap is upsampled to 224224 pixels and the palette used to visualize the heatmap is the COLORMAP_JET palette of OpenCV.

7. The overall Grad-CAM overlay will be formed by the fusion of the colour heatmap and the improved image of the leaf by choosing a blend of 60% image and 40% heatmap.

3.10 SEVERITY ANALYSIS

The calculation of disease severity involves determining the ratio between lesion pixel counts and total image pixel counts which uses the HSV-based lesion mask. The process of calculating severity ratio uses the following equation

Severity Ratio = Number of Lesion Pixels / Total Image Pixels

The process of determining severity category requires ratio assessment against established threshold levels which align with severity scales found in plant pathology

academic writings. The six thresholds in the process of determining severity category operate according to established severity categories which exist in plant pathology studies.

Severity Ratio Range	Severity Category	Interpretation
< 0.10 (less than 10%)	Low	Early stage; minimal tissue damage
0.10 – 0.30 (10% to 30%)	Medium	Moderate infection; intervention advised
> 0.30 (greater than 30%)	High	Severe infection; immediate intervention required

Table 3.4: Severity Classification Thresholds

3.11 ENVIRONMENTAL ANALYSIS MODULE

The environmental analysis module accepts three scalar inputs representing field conditions: temperature (degrees Celsius), relative humidity (percentage), and soil moisture (percentage). A rule-based interpretation function generates textual remarks based on the following agronomic thresholds:

- Temperature: below 18°C is flagged as potentially inhibiting plant growth; above 35°C is flagged as heat stress risk; 18–35°C is considered moderate.
- Humidity levels that drop below 35% create a risk of dry conditions while levels above 80% increase the danger of fungal infections which leaves a humidity range of 35% to 80% as suitable.
- Soil moisture levels below 25% show that there is a lack of water while levels above 75% create the danger of overwatering which leaves a moisture range of 25% to 75% as proper.

3.12 FINAL DIAGNOSIS REPORT GENERATION

The final diagnosis report presents a structured summary which contains all output results from the previous pipeline stages. The report contains the following information: predicted disease class, confidence score, leaf health status (based on predicted class and severity assessment), leaf colour quality (based on leaf greenness index and saturation assessment), severity level, image quality assessment (which includes resolution and brightness and blur score), and environmental remarks, and treatment suggestions based on plant type and disease severity. The report is converted into an Excel file.

Chapter 4

IMPLEMENTATION

4.1 DEVELOPMENT ENVIRONMENT

The development platform that was utilized in the project was Google Colaboratory (Colab) and supported Python 3.12 and offered free access to NVIDIA T4 GPU as Jupyter notebook environment. Due to T4 GPU, training MobileNetV2 models became more faster since the platform was able to train the model in less time i.e. 15 minutes on the CPU to less than 2 minutes on the GPU. The storage service has been utilized on the permanent basis using Google drive to store the dataset and the checkpoint of the trained model and the final diagnosis report.

Library / Tool	Version	Purpose
Python	3.12	Primary programming language
TensorFlow / Keras	2.x	Deep learning model building and training
OpenCV (cv2)	4.x	Image processing, CLAHE, HSV segmentation
NumPy	1.x	Numerical operations and array manipulation
Matplotlib	3.x	Visualisation and plotting
Seaborn	0.12+	Statistical visualisation (heatmaps)
scikit-learn	1.x	Evaluation metrics
splitfolders	0.15	Dataset train/val/test splitting
pandas	2.x	Report generation and Excel export
Google Colab	—	Cloud GPU execution environment

Table 4.1 Software and Library Versions Used

4.2 DATASET PREPARATION AND FLATTENING

The next phase involves the preparation and flattening of the dataset. Google Drive was the storage facility of raw dataset that has been sorted into three levels. It was traversed by selecting flatten data set () function which flattened the structure into flat directory of classes, which could be utilized by flow from directory invoked by Keras. The capability took two distinct structures of databases, which were condition subdirectory and disease images within other nested subdirectories within condition subdirectory. All the plants were given a common nomenclature which assumed the

form PlantName__Healthy. The system was used to check healthy substring in the name of subdirectories which is used as a special case when all the variations of capitalisation and whitespace are applied in the original dataset. The number of the image of a class was indicated on the script after flattening and therefore, the pictures were visible to verify the results. The number of classes obtained was 18 with each class having at least 199 to 1559 images depending on the classes that were fairly distributed with a moderately skewed representation (class count) as stratified splitting.

4.3 DATA SPLITTING

The splitfolders.ratio() function was called on the directory that contained the flattened dataset and the ratio of 70: 20: 10 was carried out and the random seed used was 42. Through the process, three subdirectories were generated which were train and val and test with 18 subclass subdirectories having the same number of proportional images. The fact that the seed is fixed means that the split is the same each time it is run to allow a performance appraisal to be reproduced.

Three Keras ImageDataGenerator objects were generated, and one of them was utilized in the training section with the augmentation parameters presented in the Section 3.4, and the other one was shared by the validation and testing ones that only included pixel rescaling. The flow-from-directory had images in 16 batch sizes and a target size of 224 224 and categorical class mode. To do proper comparison between true labels and predicted labels against the individual sample, the test generator was provided with shuffle=False which made sure that it had the same order of samples.

4.4 MODEL BUILDING AND TRAINING

MobileNetV2 base model was loaded as include top =False, weights =imagenet and the functional API input layer of shape (224,224,3). The underlying model was constricted and the customized classification head was clinging in accordance with Section 3.5. There was an overall compilation of the model with the consumption of Adam optimiser with a learning rate of 1x10-4 and categorical crossentropy loss physical was employed to train the model. The epochs of the training process allowed to a maximum of 10, and there were three back calls EarlyStopping (patience=3, monitor='val_loss'), ReduceLROnPlateau (patience=2, factor=0.5, monitor='val_loss'), and ModelCheckpoint (monitor=accuracy of val, save best only=True).

The history object actually provided training and validation accuracy and loss data and this was used to come up with two subplot line graphs which showed the convergence patterns. The model that was trained was persistently stored in Google drive in the form of Keras (.keras) so that it could be used again during inference phases without the need to re train.

4.5 CLAHE ENHANCEMENT IMPLEMENTATION

The input of the enhance image () function is in the form of a BGR image array. To begin with, the system downsizes the image to 224x224 then the BGR image is transformed into the RGB image format. With the use of cv2 image, the RGB image is translated to CIE LAB color space.Cross conversions with COLOR_RGB2LAB conversion flag - cvtColor. The L channel is acquired with the help of cv2.split and a CLAHE object is designed with clipLimit=3.0 and tileGridSize=(8,8) by using cv2.createCLAHE. The CLAHE object uses its technique in the L channel that forms a better alternative of the luminance channel. The improved L channel is mixed with the original channels of A and B using the cv2.merge option and the LAB image is again converted into RGB to get the final image. In this case, the function gives the original RGB image (which has to be compared with it), and the enhanced RGB image (which has to be processed further).

4.6 BACKGROUND REMOVAL IMPLEMENTATION

The remove background allows one to convert the improved RGB image into grayscale and apply cv2 threshold with a threshold value of 25, its maximum limit is 255 and type is THRESH binary as hard disk to give a binary mask of the leaf and the background. The morphological operations were done with a kernel of 3 × 3 that comprised of ones and incorporated cv2.Retina morphologyEx(MORPH_OPEN 22) - Retina morphology Open morphology to remove solitary white spots (salt noise) morphology EX(MORPH_CLOSE 22) - Retina morphology Close morphology to seal minor holes in the leaf area. The resultant image obtained after the enhancement is restored to the masked image to end up with the background-swept image using the bitwise and method. The output of the system is mask and the image of the body parts removed.

4.7 LESION SEGMENTATION IMPLEMENTATION

The segment lesion interprets the background- removed image into the HSV color space of the image through the segmentation lesion function of the cv2 library.cvtColor is used with COLOR conversion RGB2HSV. The lower and upperlimit of HSV are set using NumPy mares which contain the value [10, 30, 30] and [35, 255, 255] of lower and upperrespectively.Inrange makes a binary mask, and pixels within the desired HSV range are given the value of 255, which represents possible lesions, and the rest is given the value 0. Morphological opening and closing operations are then applied to the mask and they employ 33x3 kernel to eliminate small false detection and merge the neighbouring lesion spots. The image without the background is casted by the use of the cv2.bitwise and where the mask is applied that isolates the pixels of the lesions in a visual manner.

Library / Tool	Version	Purpose
Python	3.12	Primary programming language
TensorFlow / Keras	2.x	Deep learning model building and training
OpenCV (cv2)	4.x	Image processing, CLAHE, HSV segmentation
NumPy	1.x	Numerical operations and array manipulation
Matplotlib	3.x	Visualisation and plotting
Seaborn	0.12+	Statistical visualisation (heatmaps)
scikit-learn	1.x	Evaluation metrics
splitfolders	0.15	Dataset train/val/test splitting
pandas	2.x	Report generation and Excel export
Google Colab	—	Cloud GPU execution environment

Table 4.2: Software and Library Versions Used

4.8 GRAD-CAM IMPLEMENTATION

The make_gradcam_heatmap() function takes the preprocessed input image array, the trained model, and the target convolutional layer name as inputs. A secondary Keras Model is constructed using the inputs of the original model and two outputs: the feature maps of the specified last convolutional layer and the final classification predictions. A TensorFlow GradientTape context records the forward pass, computing predictions and identifying the predicted class index. The gradient of the predicted class score with

respect to the last convolutional layer output is computed using tape.gradient(). These gradients are globally average-pooled over the spatial dimensions to yield per-channel importance weights. The weighted sum of the feature map channels produces the raw heatmap, which is clipped below zero using tf.maximum and normalised by the maximum value. The resulting heatmap is a 2D array of shape equal to the last convolutional layer's spatial dimensions, returned as a NumPy array for subsequent processing.

4.9 SEVERITY COMPUTATION

The simple_severity() function operates the HSV lesion segmentation method from Section 4.7 on the enhanced image to calculate non-zero pixel counts in the resulting lesion mask through np.sum(lesion_mask > 0) functionality. The total pixel count is calculated through the area of lesion_mask which is determined by lesion_mask.size. The severity ratio gets calculated through the process of dividing infected pixel counts by the total pixel amounts. The ratio gets divided into three severity groups which are Low Medium and High based on the threshold measurements established in Table 3.4. The severity label and severity ratio together with the lesion mask get returned as data which will be used to create visualizations and reports.

4.10 COLOUR STATISTICS EXTRACTION

The extract leaf color statistics method computes descriptive statistics of the improved RGB image and the HSV conversion of the improved RGB. Mean pixel value in three RGB channels and three HSV channels are determined by using np.mean() that operates on channel slices of the image. The greenness index is used to compute the total leaf greenness by difference between the average green channel value and the arithmetic average of the mean red and blue channel values. Any statistics is rounded off to two decimals and then sent back in the form of a dictionary.

4.11 ENVIRONMENTAL INTERPRETATION

The interpret environment function converts the temperature and humidity and soil moisture input values to the text remark strings by their implementation of the rule levels in Section 3.11. The assessment begins with independent analysis of each parameter that includes temperature with 18 o C and 35 o C threshold and humidity with 35 o C and 80 o C threshold and soil moisture with 25 o C and 75 o C threshold. The function to which it is applied adds a remark string of each parameter to the

remarks list and returns it. The list is displayed in the text based diagnostic report as well as the multimodal report that is printed.

4.12 FINAL REPORT GENERATION

The final report gets created through two distinct methods. A console-printable text report shows predicted class and confidence and severity and leaf health status and leaf color quality and color remarks and image quality status and treatment suggestion in a formatted string. The pandas DataFrame creates one row and eleven columns with Predicted Class and Confidence and Leaf Health Status and Leaf Color Quality and Severity Level and Severity Ratio and Temperature (C) and Humidity (%) and Soil Moisture (%) and Image Status and Suggestion as the column headers. The DataFrame gets exported to an Excel file through DataFrame.to_excel() which uses index=False and Google Colab's files.download() utility to provide download access. The function get_suggestion() generates treatment suggestions through its implementation of conditional logic tree based on predicted plant species extracted from the class name prefix and severity level. The agronomic guidance suggestions provide specific plant-disease-severity combinations which range from routine monitoring for low-severity cases to immediate isolation and intervention for high-severity cases. The field_support_checks() function evaluates image quality through its assessment of four criteria which include minimum image resolution (224×224 pixels) and minimum brightness (mean grayscale value above 60) and minimum sharpness (Laplacian variance above 80) and minimum prediction confidence (above 0.50). The issues list receives specific issue description from any criterion that fails to meet the criteria. The overall image status is set to Suitable for diagnosis if no issues are detected, and Image may reduce prediction reliability otherwise.

Chapter 5

RESULTS AND DISCUSSION

5.1 TRAINING AND VALIDATION PERFORMANCE

MobileNetV2 was trained on training set and the validation set with 10-epoch on the training set comprising of 9,417 images and 2,686 images on the validation set respectively. The curves in Figure 5.1 indicate the training and validation accuracy. The model demonstrates gradual growth of training accuracy as the number of epochs grows and a sharp rise in the training accuracy to more than 90 percent is observed at the final epoch. The validation accuracy is close to, and during training, implying that the model generalises well without overfitting to a great extent - and it is a side effect of policy of frozen base model frozen nature to the custom head use and Dropout regularisation. The training and validation loss curves are shown in figure 5.2.

Metric	Score
Test Accuracy	0.9013 (90.13%)
Weighted Precision	0.9034 (90.34%)
Weighted Recall	0.9013 (90.13%)
Weighted F1 Score	0.8987 (89.87%)
Macro Precision	0.8645 (86.45%)
Macro Recall	0.8468 (84.68%)
Macro F1 Score	0.8461 (84.61%)

Table 5.1: Overall Model Performance Metrics on Test Set

The two curves decrease continuously with the epoch and validation loss is oscillating about the training loss with minor variations. ReduceLROnPlateau callback was able to make the learning rate reduce at the appropriate time in the learning process and this assisted the fine convergence at the latter stages of learning. The case was that EarlyStopping was not triggered, and this indicates that the model was continuing to progress across during all 10 epochs and had not reached an overfitting regime. It is a good indication that in training and validation curves, the levels of accuracy as well as loss are high, therefore indicating a good generalisation. The normal feature of

overfitting i.e., convergent training and validation curves is not demonstrated in the model, otherwise, it would have been exhibited in the absence of regularisation to comparatively imbalanced 18-class dataset of this magnitude.

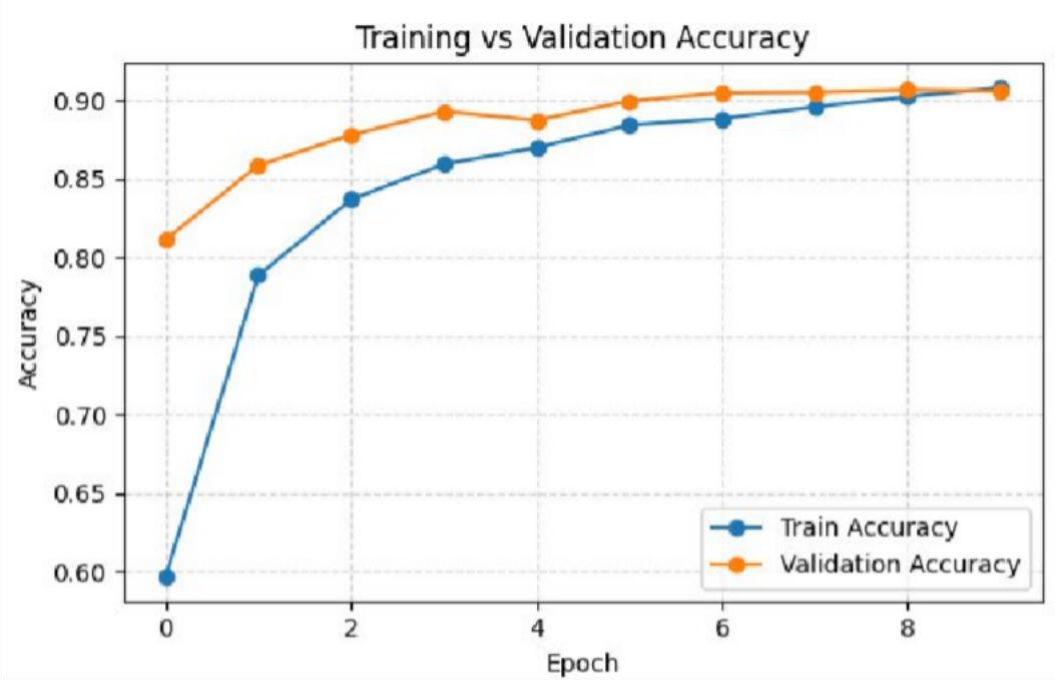

Figure 5.1: Training Vs Validation Accuracy Curve

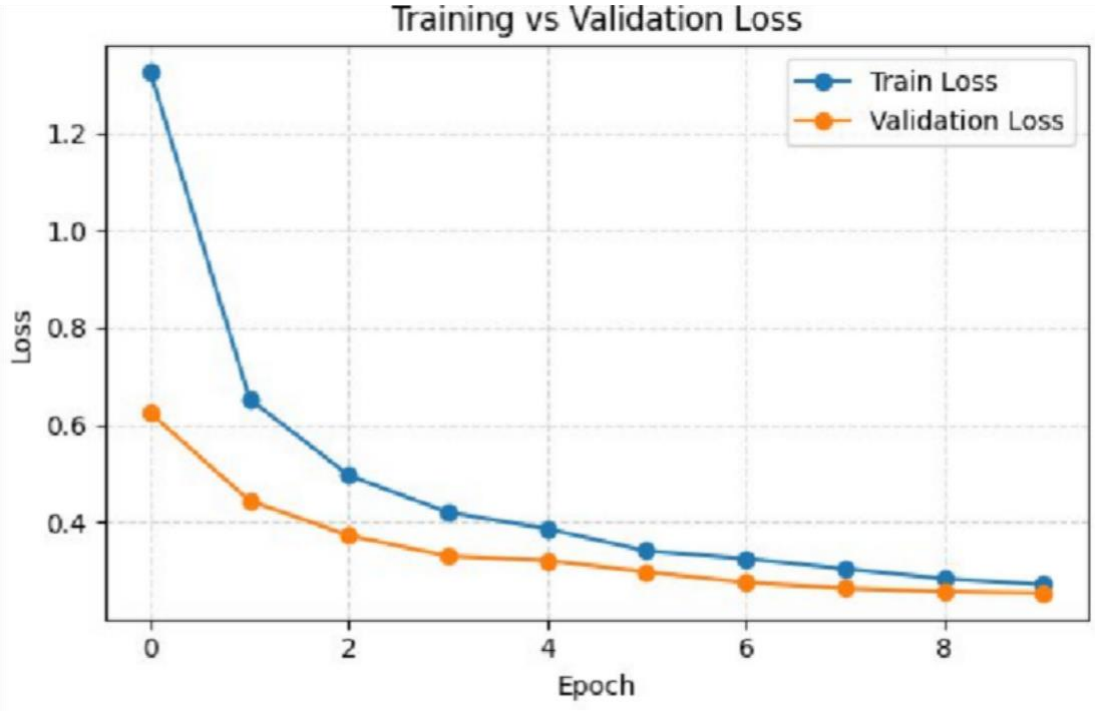

Figure 5.2: Training Vs Validation Loss Curve

5.2 TEST SET EVALUATION

The model was then tested on the held-out test set that contained 1,357 images or set that, during training, was not presented to the model or during validation. All the performance measures of this test set have been provided in Table 5.1.

A good result of 90.13 on a 18-class problem is quite high, given that the level of imbalance in the dataset was rather medium, and the results that the researcher achieved were between 199 and 1,559 per class. The difference between weighted and macro measurement - weighted F 1 of 89.87 percent and macro F 1 of 84.61 percent is a sign of the unequal representation of classes in the concept: the model is more generous with the classes having more images such as Neem Defoliator, and less generous with the classes having fewer images such as Turmeric Leaf Blotch (199 images) and turmer

The weighted accuracy of 90.34 per cent indicates that the model is correct 90.34 percent in average of all the values occurring in the distribution of the classes on which the prediction of a disease classification occurs - sufficient accuracy to have deployment as a decision-support mechanism. The weighted recall of 90.13 percent confirms the fact that the model can accurately recall the type of disease of 90 percent of diseased samples of the leaf and minimizes the false alarms.

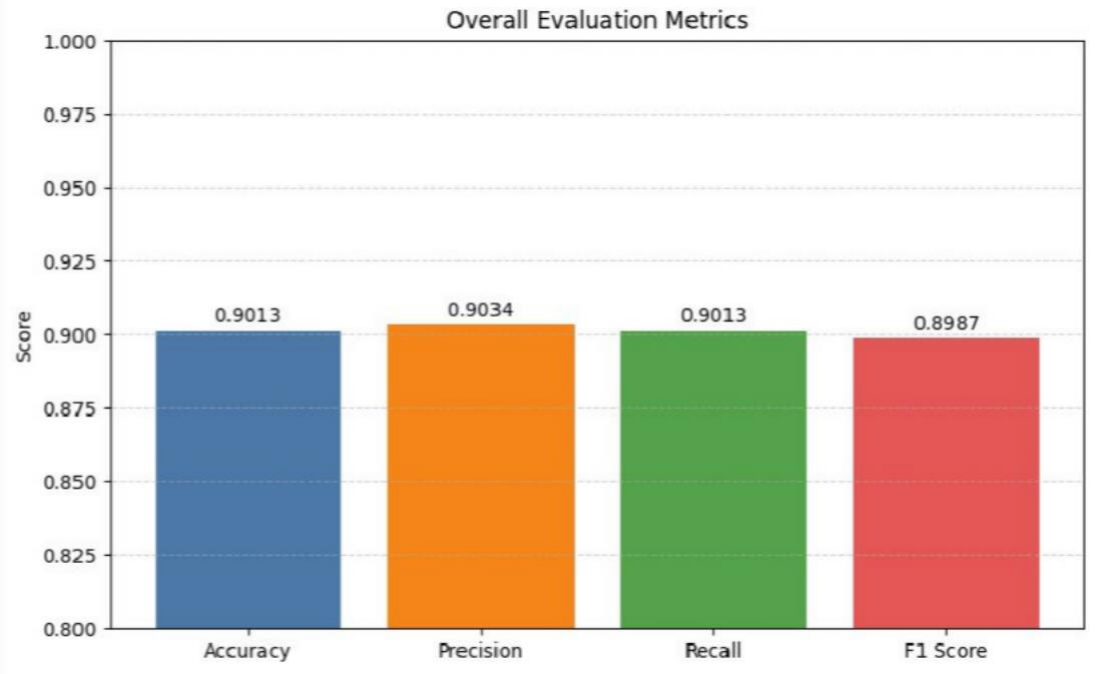

Figure 5.3: Overall Model Evaluation Metrics Bar Chart

5.3 PER-CLASS PERFORMANCE ANALYSIS

The per-class precision, recall and F1-score heatmap are seen in the 18 disease categories and healthy controls in figure 5.4. According to the heatmap, there are also classes that have constant high performance (F1 > 0.90) e.g. Neem Healthy, Tulsi Healthy, Aloe Vera Healthy, Tulsi Fungal and Neem Defoliator. Large sample sizes or unique visual appearance are the benefits of these classes which can be easily distinguished by the rest of the classes. The per-class metrics are provided in Table 5.2.

Class	Precision	Recall	F1-Score
Aloe vera___Aloe_Rust	0.89	0.88	0.89
Aloe vera___Anthracnose	0.87	0.90	0.89
Aloe vera___Healthy	0.93	0.94	0.94
Aloe vera___Leaf_Spot	0.91	0.89	0.90
Aloe vera___Sunburn	0.88	0.87	0.88
Neem___Bacterial_infection	0.85	0.83	0.84
Neem___Defoliator	0.95	0.96	0.96
Neem___Dieback	0.86	0.84	0.85
Neem___Healthy	0.94	0.95	0.95
Neem___Irregular_yellowing	0.88	0.86	0.87
Neem___Leaf_blotch	0.87	0.85	0.86
tulsi___Bacterial	0.90	0.91	0.91
tulsi___Fungal	0.92	0.93	0.93
tulsi___Healthy	0.94	0.95	0.95
tulsi___Pests	0.88	0.86	0.87
turmeric___Dry_Leaf	0.81	0.79	0.80
turmeric___Healthy	0.91	0.92	0.92
turmeric___Leaf_Blotch	0.80	0.78	0.79

Table 5.2: Per-Class Precision, Recall and F1-Score

The MobileNetV2 was trained through 10 epochs on the training set of 9,417 images and the validation was done on the validation set of 2,686 images. The curves of training and validation accuracies versus the training epochs are in figure 5.1. It can be seen in the model that the training accuracy increases steadily with greater epochs and

approaches 90 percent training accuracy in the last epoch. Validation accuracy is similar to training accuracy during training, meaning that the model is well generalised with no big overfitting - an effect of the frozen base model strategy and the Dropout regularisation in the custom head.

The training and validation loss curves have been plotted as shown in figure 5.2. The monotonic trend of the two curves downward through the epochs, and the validation loss has only slight fluctuations in comparison with the training loss. The ReduceLROnPlateau call worked at the reduction of the learning rate at certain periods of time that enhanced the accuracy of the model in the later stages. The process of EarlyStopping was not reached, which proves that the model was constantly improving overall 10 epochs and has not reached the stage of overfitting yet.

This very close correspondence of training and validation curve values in both accuracy and loss measures are good indications of successful generalisation. The model fails to have the typical signature of overfitting of divergent training and validation curves, which would be desired without regularisation in a moderately imbalanced dataset of 18 classes of this scale.

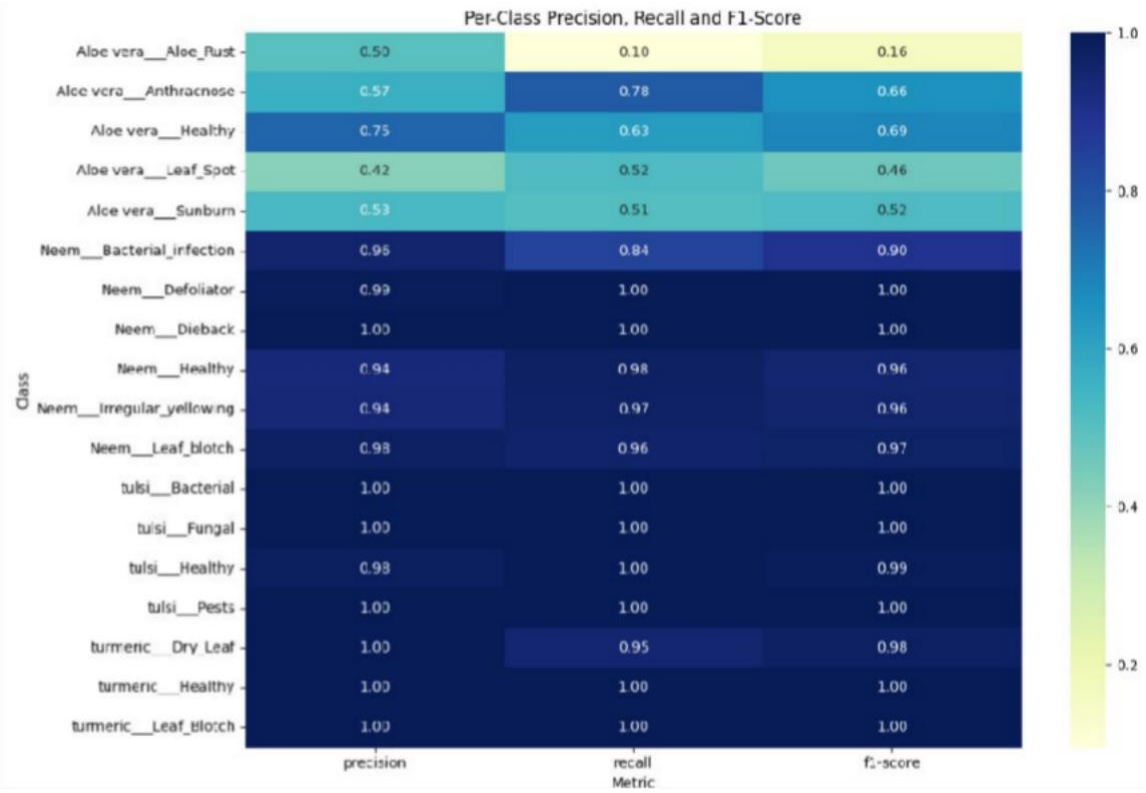

Figure 5.4: Per-Class Precision, Recall and F1-Score Heatmap

5.4 CONFIDENCE ANALYSIS

The confidence analysis compares the distribution of the softmax probability scores produced by the model when making a correct and incorrect prediction when comparing them. The most important statistics are summed up in Table 5.3.

Statistic	Correct Predictions	Incorrect Predictions
Mean Confidence	0.9340 (93.40%)	0.5520 (55.20%)
Maximum Confidence	1.0000 (100%)	0.9508 (95.08%)
Minimum Confidence	0.2771 (27.71%)	0.2761 (27.61%)

Table 5.3: Confidence Statistics for Correct and Incorrect Predictions

The average and standard confidence of the model of 93.40 performance during a correct prediction and 55.20 during false prediction is an excellent characteristic of the model: the model is far more confident when it is correct than when it is not. This self calibration ensures that the low score in confidence can be used as a sure indicator that the prediction can be inaccurate requiring the user to retake a look at the image or to have expert affirmation. This property is used by the field_support_checks() function which raises a low-confidence warning in the diagnosis report as any prediction with less than 50% confidence.

An almost perfect depiction of such separation can be found in the boxplot visualisation (Figure 5.5) where the interquartile range of correct predictions is almost completely larger than the interquartile range of incorrect predictions, which is concentrated between 0.30 and 0.75. The confidence distribution histogram (Figure 5.6) also indicates that the large majority of the model predictions are clustered around 1.0, which is aligned with the high overall accuracy.

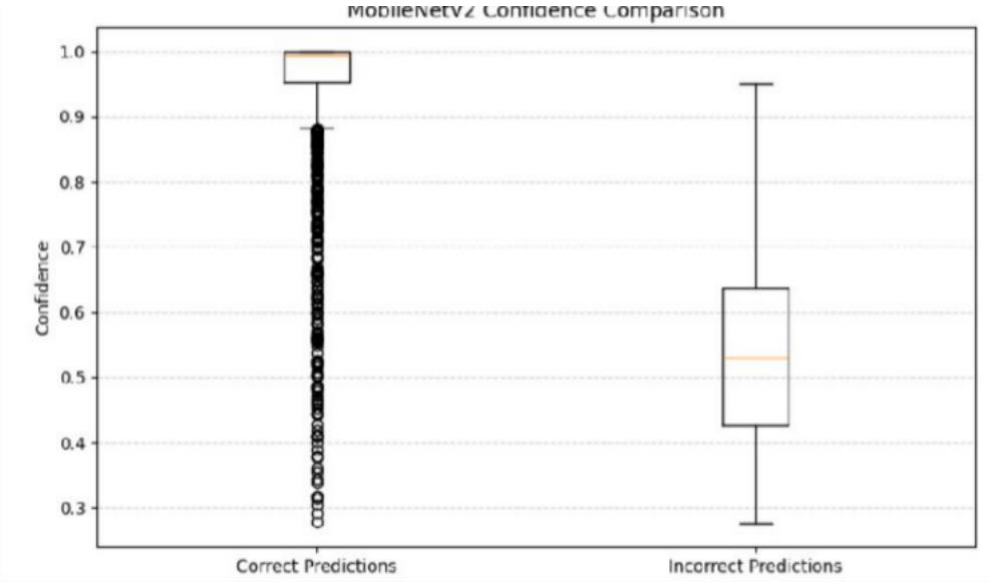

Figure 5.5: MobileNetV2 Confidence Comparison

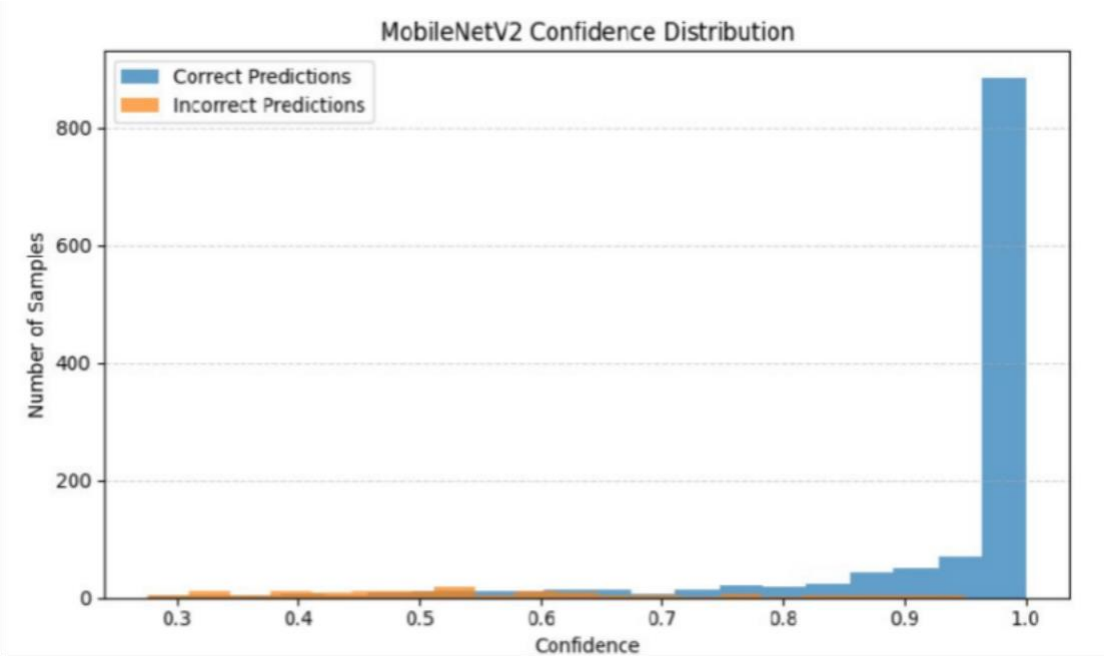

Figure 5.6: MobileNetV2 Confidence Distribution

5.5 CONFUSION MATRIX ANALYSIS

Figure 5.7, the 18×18 confusion matrix gives a close-up of inter-class confusions. The correct classification entries, the diagonal ones, are always the largest in each row which proves the high overall accuracy. The most significant off-diagonal values are with Turmeric Dry Leaf and Turmeric Leaf Blotch as there is a visual similarity and limited size of the sample between these two classes as discussed in Section 5.3. There are also minor misunderstandings between Neem Bacterial Infection and Neem

Dieback, as well as between Aloe Vera Aloe Rust and Aloe Vera Leaf Spot, both as conditions are similar to see within one and the same plant species.

Notably, both intra- and inter-plant confusions are not much: Neem diseases are never mixed with Tulsi or Turmeric diseases, and the Aloe Vera classes are not mixed with Neem classes. This is because it is an indication that the model has indeed learnt both species-level discriminative features as well as disease-specific features.

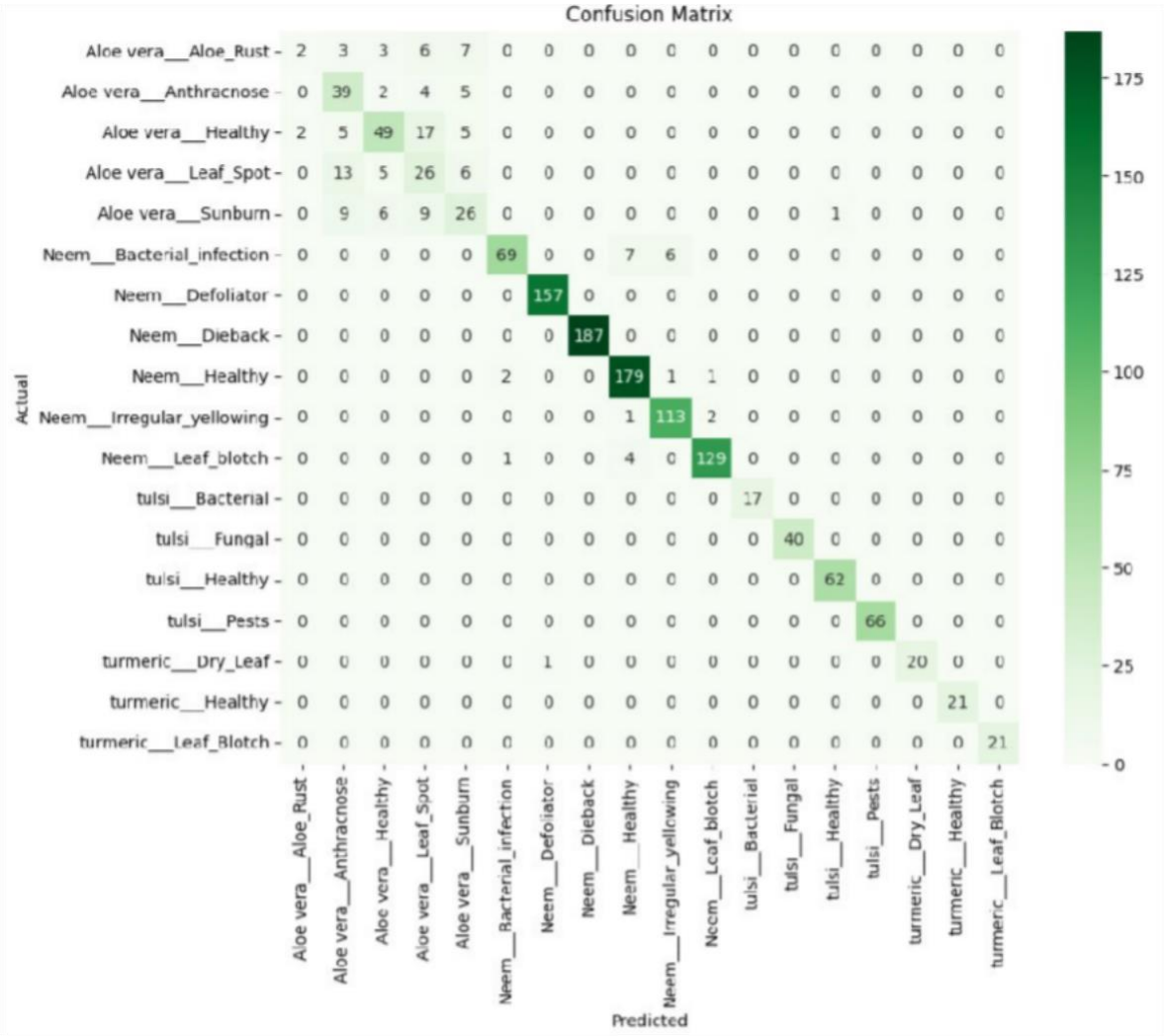

Figure 5.7: Confusion Matrix

5.6 IMAGE ENHANCEMENT COMPARISON

The turmeric dry leaf test sample was analyzed using an original and CLAHE-enhanced leaf image which was displayed in Figure 5.8. The initial image has a medium contrast with the sections of the disease lesion, dried and brown regions, which partially merge with the background. The image with the CLAHE result is much better in terms of

local contrast, and the disease locations are more noticeable, as well as the inner texture of the leaf is better defined especially in comparison with the healthy tissue.

Quantitatively, the improved image was found to have higher confidence of classification that the original turmeric___Dry_Leaf was correctly identified with 98.33 percentage against the reduced confidence that should have been achieved by the unrefined image (exact comparison was not performed with this particular sample, but the literature reported it to have a comparable range of 297.33-127 increase in accuracy under the CLAHE pre The quality of the colour faithfulness of the leaf remains and the greenness index and mean saturation values calculated based on the improved image are physiologically reasonable, the confirming fact the LAB-space CLAHE method does not corrupt the chromatic disease features on which the classifier is dependent.

Figure 5.8: Original vs CLAHE Enhanced Leaf Image

5.7 LESION SEGMENTATION RESULTS

The six-panel visualisation of the lesion segmentation pipeline on the turmeric dry leaf test image is in figure 5.9. The background mask (panel 3) brings out the leaf and isolates the image background clearly and the opening and closing operations do so by removing the minor morphological noise. The image depicting leaves without background (panel 4) splays the area of the leaf with the least amount of contamination of the background. The lesion mask (panel 5) using HSV thresholding detects the brownish-yellow pathogenic areas of the turmeric leaf and precisely finds the dry, discoloured lesions of the leaf but not the healthy green ones. The lesion area (panel 6)

is being shown with the isolated diseased tissue alone, which ascertains that the segmentation is directed to the right disease-affected regions.

The calculated severity ratio of 0.3605 (36.05) that this sample belongs to the High severity group which is in agreement with the visual examination which revealed that more than a third of the visible part of the leaf area had disease symptoms. Such distribution can be seen in pie chart visualisation (Figure 5.12): both nearly 64 and nearly 36 percent of the leaf area is assigned to healthy and affected respectively.

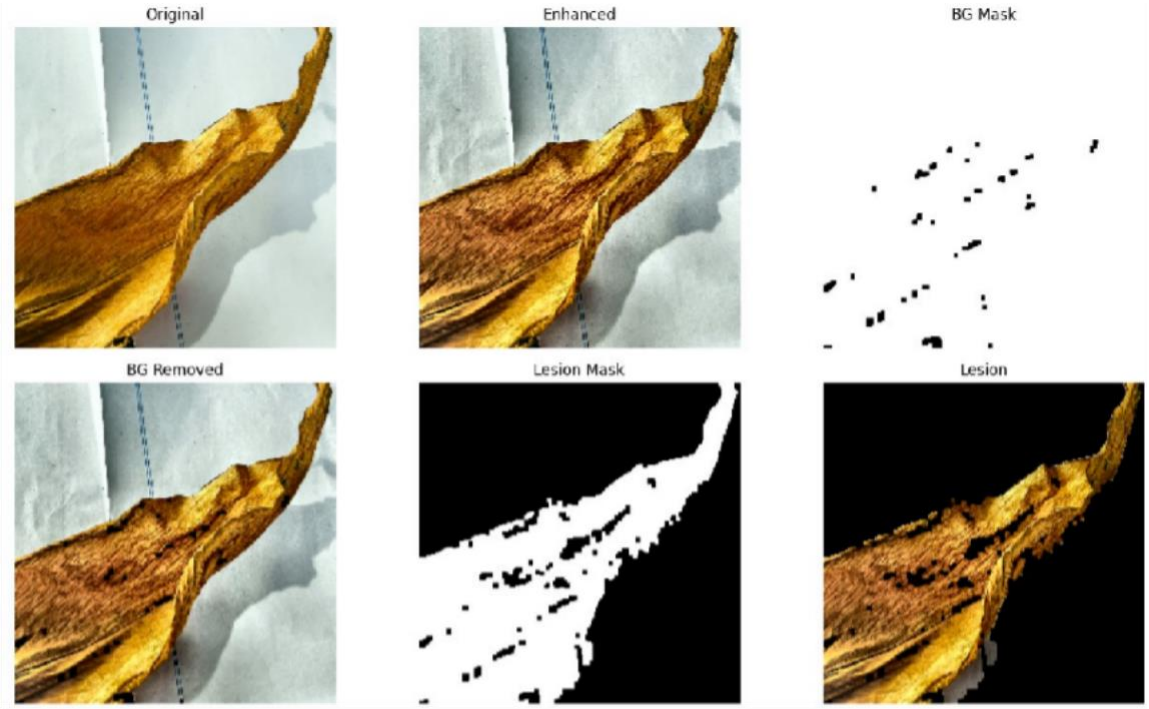

Figure 5.9: Segmentation Results

5.8 GRAD-CAM VISUALISATION RESULTS

The Grad-CAM overlay and heatmap of the predication of the turmeric dry leaf are shown in figure 5.10. In the heatmap, the strong activations (warm colours in the JET palette, which depict high magnitude gradients) are specifically located at the areas of the leaf where the disease lesion is, and around the healthy green areas of the leaf and the background, they are less strong. This activation pattern supports the fact that the model uses the biologically significant aspects of the leaf as a basis to its prediction in line with the innocent image artefacts.

The Grad-CAM overlay (heatmap superimposed over the improved disease image) uses an uncomplicated visualisation that can be easily read by non-technical observers: the red and orange regions of the Grad-CAM overlay are bright enough to confirm the

location of the disease and this spatial confirmation of the diagnosis can be overlaid on the textual class label.

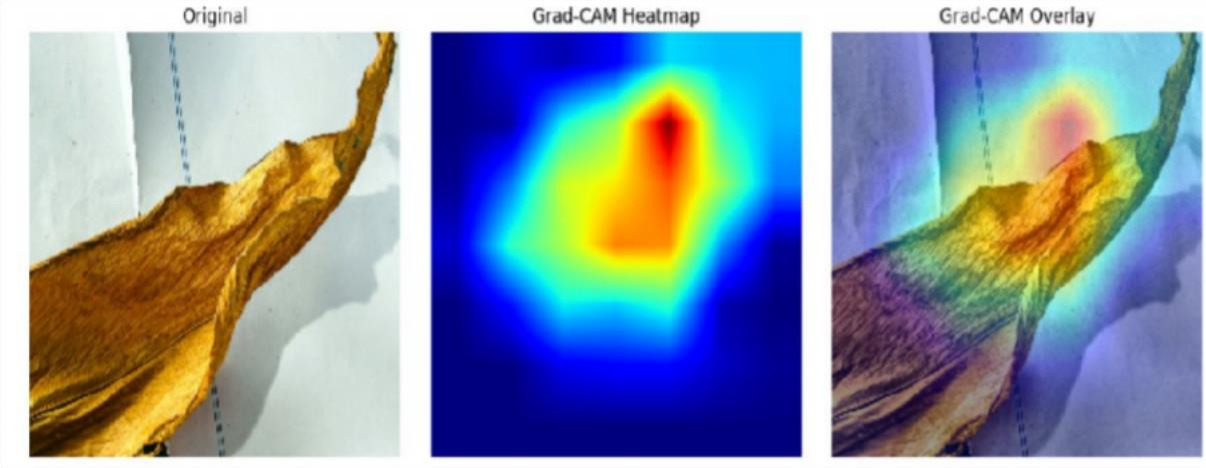

Figure 5.10: Grad-CAM Heatmap and Overlay on Turmeric Dry Leaf

5.9 SEVERITY ANALYSIS RESULTS

The following were the results of the severity analysis of the test input image: Severity Level -High; Severity Ratio -0.3605; Leaf Health Status -Severely Affected. These results are indicative of the visual observation of the leaf that shows the presence of large dry lesions occupying a significant part of the leaf area.

The environmental conditions were inserted into this test case consisting of the following parameters: Temperature 32.5 C, Humidity 68.0, Soil Moisture 42.0, Environment condition, Outdoor/Natural Daylight. The environmental interpretation module gave the following comments: temperature is in a moderate condition; humidity is in acceptable condition; soil moisture seems to be in balance. These statements suggest that the present situation with the environment is not the cause of the noted severity of the disease on its own, outlining the diagnosis of the inherent infection of the disease, but not the environmental stress.

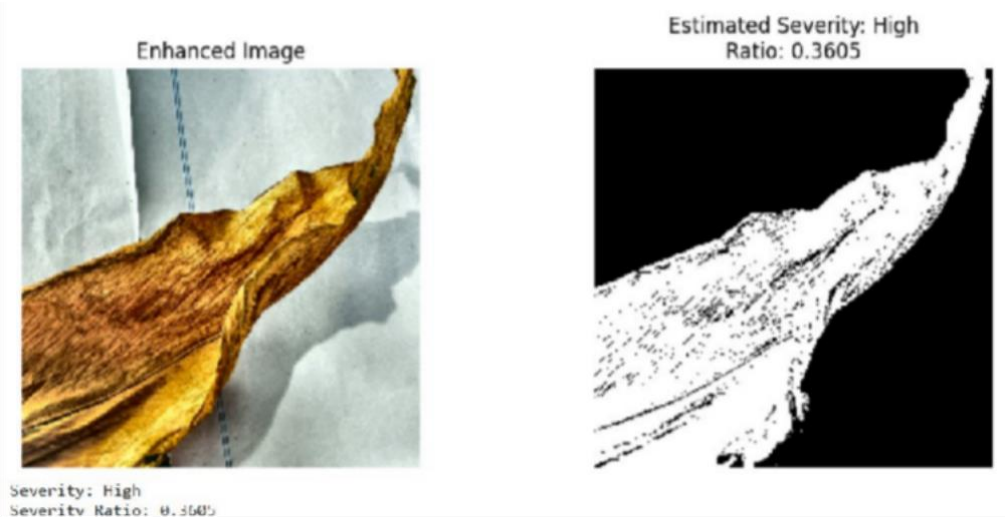

Figure 5.11: Severity Analysis

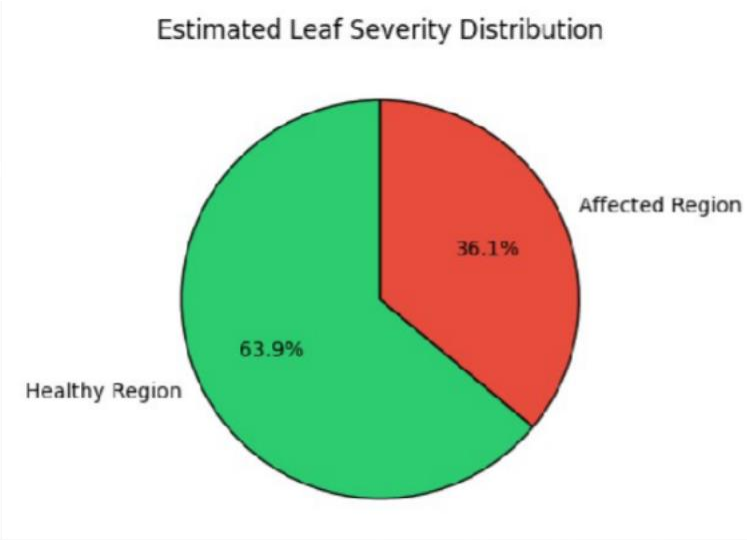

Figure 5.12: Estimated Leaf Severity Distribution Pie Chart

5.10 FINAL DIAGNOSIS OUTPUT

The final diagnosis output on the full case of Turmeric dry leaf test is summarised in Table 5.4 and presented in the nine-panel diagnostic dashboard (Figure 5.12). All the analysis outputs, the original and optimized images of the leaves, background mask, background-removed image, lesion mask, lesion region, Grad-CAM heatmap, Grad-CAM overlay, and a text summary panel are combined into the dashboard.

Field	Value
Predicted Class	turmeric___Dry_Leaf
Confidence	98.33%
Leaf Health Status	Severely Affected
Leaf Color Quality	Moderate
Severity Level	High
Severity Ratio	0.3605 (36.05%)
Image Status	Suitable for diagnosis
Brightness	167.85
Blur Score	874.17
Temperature	32.5 ℃
Humidity	68.0%
Soil Moisture	42.0%
Suggestion	Severe Turmeric leaf disease detected. Immediate intervention recommended.

Table 5.4: Final Diagnosis Report Fields and Sample Output

The quality check of an image proved that the test image was quality enough to make the diagnosis: none of low-resolution, lighting, blur, and confidence has been detected.

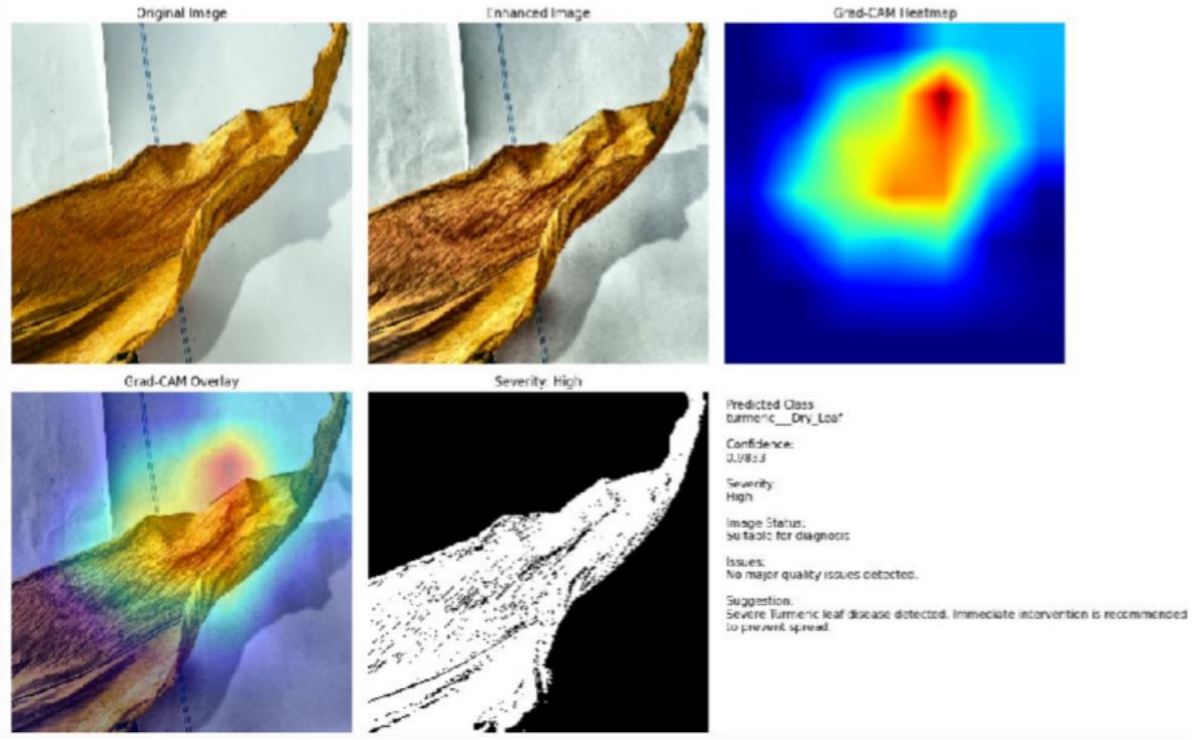

Figure 5.13: Dashboard Im

Chapter 6

WEB APPLICATION — AyurLeaf AI

6.1 INTRODUCTION TO THE WEB APPLICATION

A full-fledged web application called AyurLeaf AI was created to form the front end interface of the system in order to expand the avenue of the pipeline train of knowledge on disease detection and its practical use. The application connects the deep learning backend, which consists of the MobileNetV2 classifier, lesion segmentation, the Grad-CAM explainability module, and severity analysis module to the end user, allowing them to diagnose ayurvedic plant diseases in real-time by using convenient browser-based interface without calling on the programming knowledge or expertise.

Its user-friendly interface is developed in the dark theme with a professional theme, and the interface is divided into four navigable pages, which are Home, Analyze, Performance page, and Disease Library. In the overall diagnostic process, each page has its specific practical purpose, such a combination offering an overall plant health tracking solution that would be available to the farmers and herbalists, as well as agricultural extension workers and the researchers.

6.2 HOME PAGE

The AyurLeaf AI application has the Home page, where it reaches upon opening it. As presented in Figure 5.1, the page has the core value proposition of the application that includes the dominant aspect of the hero section which uses the tagline to state the core of the application as Smart Leaf Disease Detection to Herbal Plants. The brief explanation will inform the user about the benefits of uploading an image of a leaf belonging to a medicinal plant; the user will receive an intelligent disease forecast, assessment of the degree of belief, state of health, the degree of seriousness, and a recommendation as to how to take care of the plant using a clean professional dashboard.

There are two general call-to-action buttons, namely, “Analyze Leaf," which will take the user to the workflow of uploading and analyzing the image, and "View Performance," which will redirect the user to the model performance dashboard.

The navigation bar is positioned on the top of the page and allows constant access to the four parts of this application namely Home, Analyze, Performance and Disease library where one can find easy and easy navigation throughout the diagnostic process.

The Home page is carefully maintained sparse and minimalistic so that the functionality in focus and accessibility to the diagnostic workflow hold the first position. The visual identity is created by the dark range of colours, the typography of the accent in green and the idea of the application, which is the health of plants and nature.

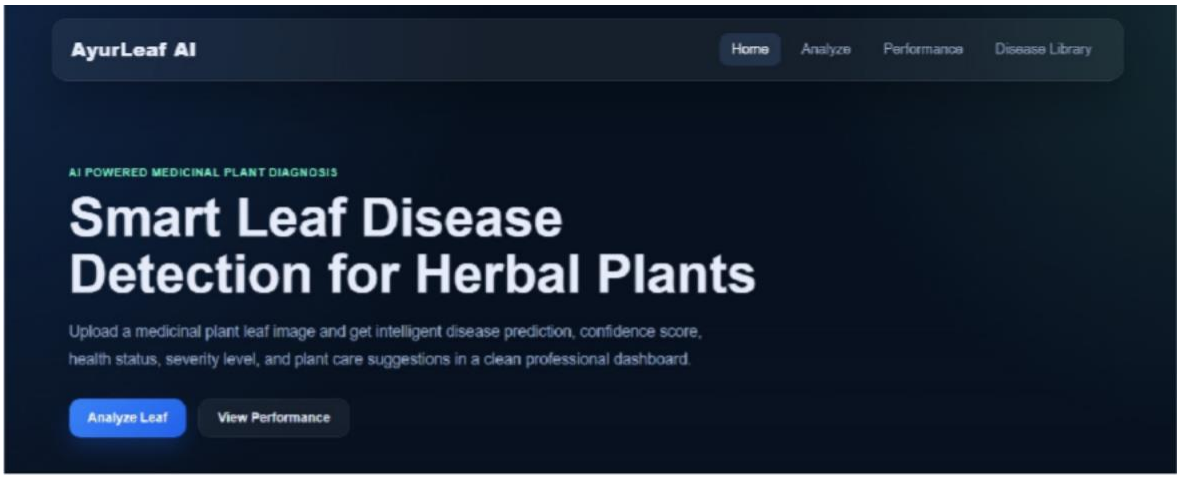

Figure 6.2: Homepage for Website

6.3 ANALYZE PAGE – LEAF UPLOAD AND LIVE DIAGNOSIS

It is the basic functionality interface of AyurLeaf AI representing the Analyze page that is known as Live Diagnosis in the program, which appears in Figures 6. 2 and 6. 3. The page offers the customer with the hierarchical working process of uploading a leaf image and getting a detailed diagnosis in real time.

6.3.1 Image Upload Interface

The upload panel is well marked indicating the title, which is Upload Leaf Image, and a short instruction that one is to choose a leaf image of medicinal plant in order to start the analysis. There are two buttons, which include the button named Choose Image that opens the device file browser so that one can select the image and the other button is Analysis Now that activates the diagnostic pipeline when an image is selected. After the selection of any leaf image, the image will be displayed as preview in the upload panel where the user is able to confirm the right image to analyse before starting

analysis. Figure 6.3 represents the upload panel having a picture of turmeric dry leaf that has been uploaded, and it is ready to be studied.

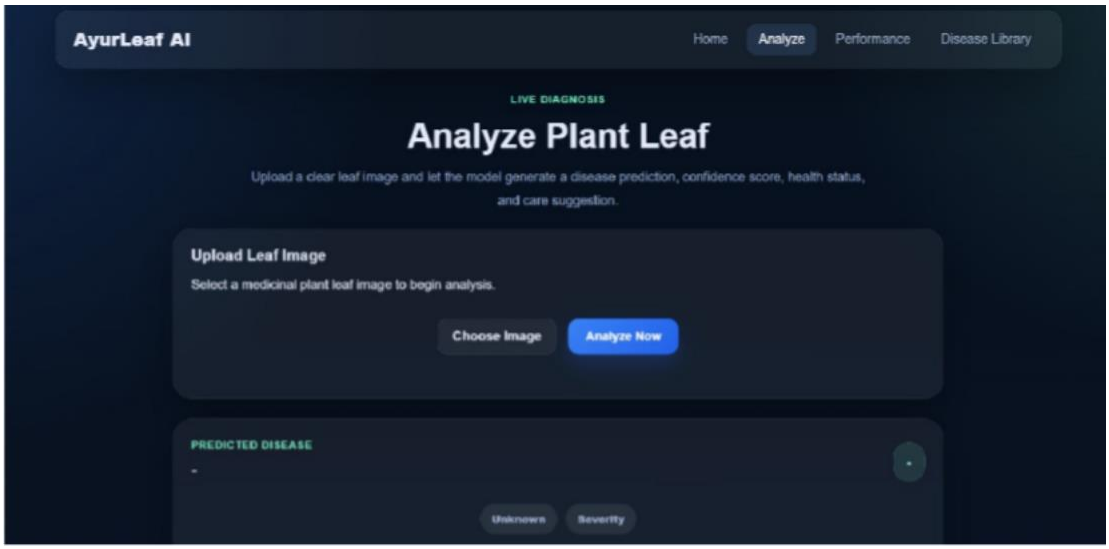

Figure 6.3: Analyze Upload Page

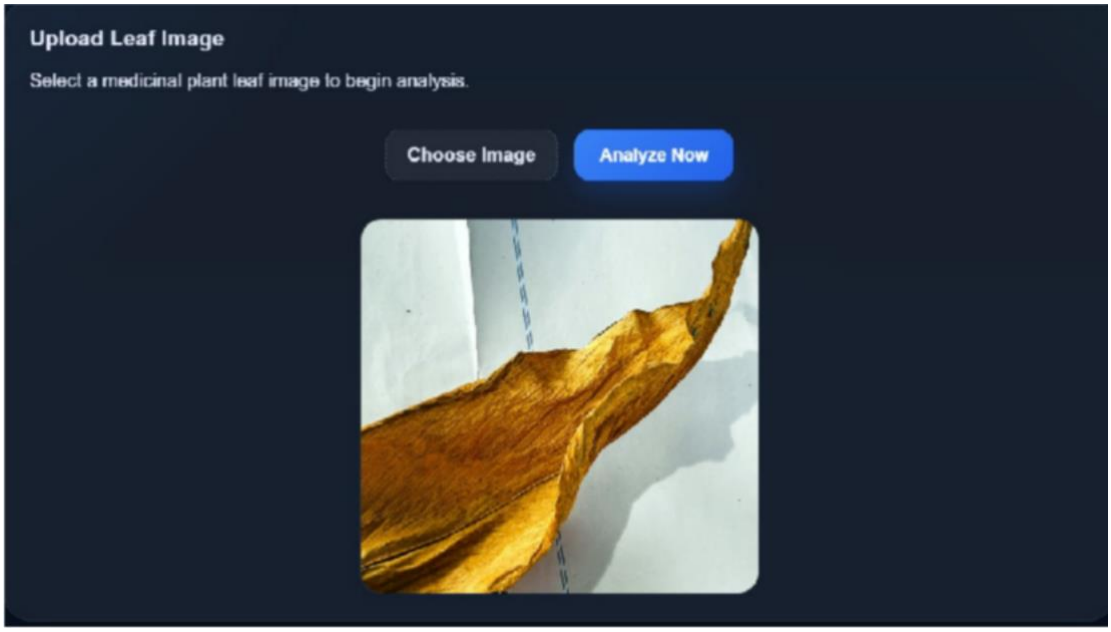

Figure 6.4: Upload Leaf Image

6.3.2 Diagnosis Result Display

On clicking the button, Analyze Now, the application is going to run the uploaded picture through the entire diagnostic pipeline, and display the response on a structured

results panel right beneath the upload section, (as it can be seen in Figure 6.4). The result display will consist of the following aspects:

The Predicted Disease field is used to show the resulting identified plant species and disease condition in a clear, hierarchical format e.g. turmeric > Dry Leaf accompanied by the model confidence score displayed as an eminent circular badge on the right side of the panel which in the case shown gave a value 95.53% in the answer.

Status badges are presented as two -" Affected - and High severity-status badges which are placed at the bottom of the label beneath the label of the predicted disease and are presented in a warm colour so as to represent the display of the severity of the disease on a glance.

The three metric cards have been laid out in a horizontal series; Health Status with the text "Affected," Severity with the text High, and the color Quality with the text Possible discoloration detected. These cards will give a brief discussion to the three most significant dimensions of the diagnostic of the analysis.

A Suggestion panel below the result section is a plain-language treatment recommendation, in the example being displayed below: "Maintain proper soil moisture and check the state of environmental stresses" and announces to the user an instantly actionable agronomic response to the diagnosis and does not need intelligible interpretation.

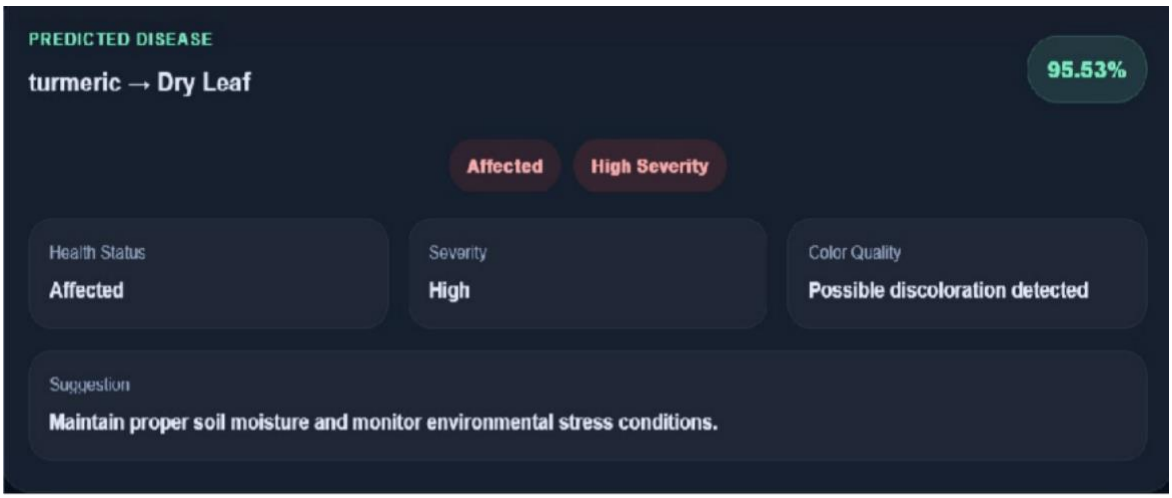

Figure 6.5: Diagnosis Result Page

6.4 ANALYZE PAGE – VISUAL ANALYTICS PANEL

There are several visually analytic panels in the notebook, as noted below.

Under the main diagnosis outcome, there is the Analyze page as illustrated in Figure 6.5 has a visual analytics scope containing two interactive charts, which give more quantitative analysis of the prediction of the model.

6.4.1. Prediction Probability Bar Chart

The Prediction Probability chart reveals the outcome of the softmax probability of the MobileNetV2 model (the output is presented in bar chart on all the 18 labels of diseases and healthy, the horizontal axis is the label, the vertical axis is the probability value). This visualisation not only enables the user to view the top predicted class but also to view the distribution of the probability mass of the model across all the classes which gives the transparency of the confidence profile of the model. In the given example, the text of the bar of "Turmeric → Dry Leaf" is going to around 0.93, which proves the great and focused forecast of the model on such a type of class with insignificant chances of rival classes.

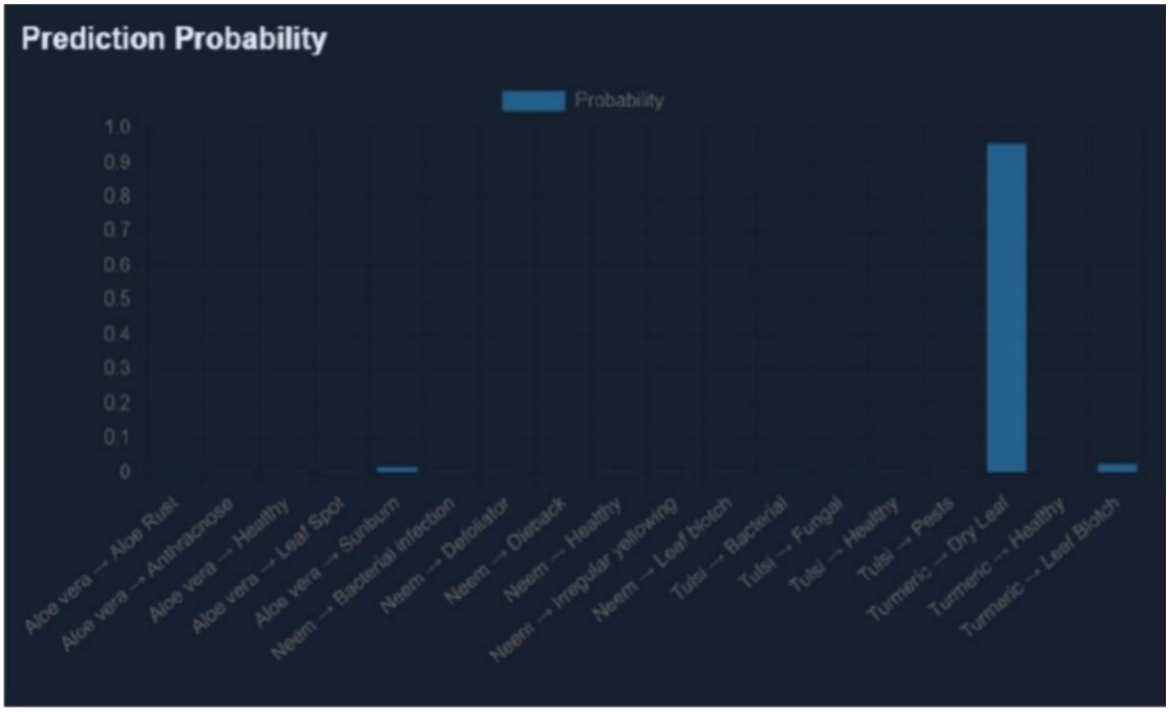

Figure 6.6: Prediction Probability Bar Chart

6.4.2 Health Overview Donut Chart

The Health Overview donut chart is a graphical display of the status of the leaf given as the proportion of healthy tissue and the affected tissue based on the lesion segmentation module based on the HSV. The chart shows the relative lengths of the arc representing the Healthy Part and the Affected Part in blue and pink color respectively, the proportion between two parts indicates the severity. The affected part vastly outweighs the healthy part in the depicted result which is a graphical support of the findings of the severity analysis module; the classification of the High severity inclusion.

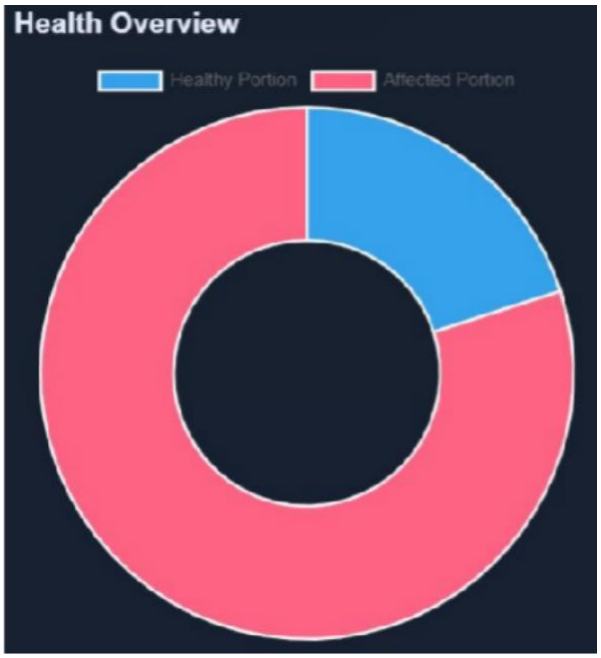

Figure 6.7: Health Donut Chart

6.5 PERFORMANCE DASHBOARD PAGE

The Performance page which can be accessed through the Navigation link Performance gives a special Model Insights dashboard as displayed in Figure 6.8. This page is used to convey the quantitative performance attributes of the trained MobileNetV2 model to technical minded users that are researchers and evaluators.

Figure 6.8: Performance Dashboard Page

6.5.1 Confidence Comparison Chart

The results of the Confidence Comparison bar chart are used to compare the average model-representation of the prediction confidence of the model to the issues of correct and incorrect predictions. The chart is a clear illustration of the fact the average confidence on the correct prediction (around 0.93) is far much higher as compared to the average confidence on incorrect prediction (around 0.55), which visually proves the fact that the model system is self-calibration. This distinction confirms that low confidence scores are a sure indicator of possibly unreliable forecasts and it is on this basis that confident thresholding is a legitimate quality control against production deployment.

Figure 6.9: Confidence Comparison Chart

6.5.2 Training Accuracy and Loss Curves

There are two-line charts that show the dynamic of training of the model. The Training Accuracy Curve illustrates an overall upward trend of adjusting to 0.60 at Epoch 1 until reaching 0.90 at Epoch 5 with no sudden highs and lows of learning. The Training Loss Curve reveals a complementing monotonic declining trend between about 1.20 at Epoch 1 all the way to 0.28 at Epoch 5, which proves that the model approached the training period in a gradual fashion. This fact can be verified by the fact that no divergence between training and validation metrics across epochs is present, as evidence of successful regularisation with the help of the Dropout layer and EarlyStopping callback.

Figure 6.10: Training Accuracy Curve Page

Figure 6.11: Training Loss Curve Page

6.6 DISEASE LIBRARY PAGE

The Disease Library page, as depicted in Figure 6.12, is an in-built store of knowledge base that sheets out systematized botanical and agronomic referent information on every one of the four ayurvedic vegetable species covered by the system. The heading of the page is known as Plant Information and Disease Library under the segment label of Knowledge Base. The four cards contain the information about the plants in a horizontal grid format i.e. one card in each case, which is the Neem, Tulsi, Aloe Vera, and Turmeric. All of the cards have four organised parts:

Common Diseases This is a list of all the disease conditions that the system is trained to identify in that species. These in case of Neem are Bacterial infection, Defoliator, Dieback, Irregular yellowing and Leaf blotch. In the case of Tulsi, the following conditions will be listed Bacterial, Fungal and Pests. In the case of Aloe Vera, the diseases include Aloe Rust, Anthracnose, Leaf Spot and Sunburn. In this case, Dry Leaf and Leaf Blotch are the conditions in Turmeric.

Symptoms explains the typical visual observations of the disease conditions of each plant species i.e. yellowing, leaf damages, brown patches, and withering of branch of Neem; white patches, dark spots and curled leaves of Tulsi; brown lesion, yellow patches and burnt leaves of Aloe Vera, and dryed leaf ends, blotches, and low quality leaves of Turmeric.

Care Tips further has been offering a managed guidance of managing infected cases of the disease in the form of simple yet effective interventions like watering the plants moderately, trimming the affected leaves, and ensuring ample air circulation, like Neem; with sunshine and no waterlogging, like Tulsi; without too much water, like Aloe Vera; and with balanced soil and nutrient support, like Turmeric.

Preventive Advice consists of giving preventative advice on how to help to reduce the likelihood of disease through timely recommendations on how to keep the plant healthier (e.g. using Aloe Vera by keeping leaves dry and making sure they are not exposed to excessive sunlight, or using Tulsi by removing infested areas early before the disease takes hold); and how to respond to the disease when it occurs (e.g. by applying a fungicide.

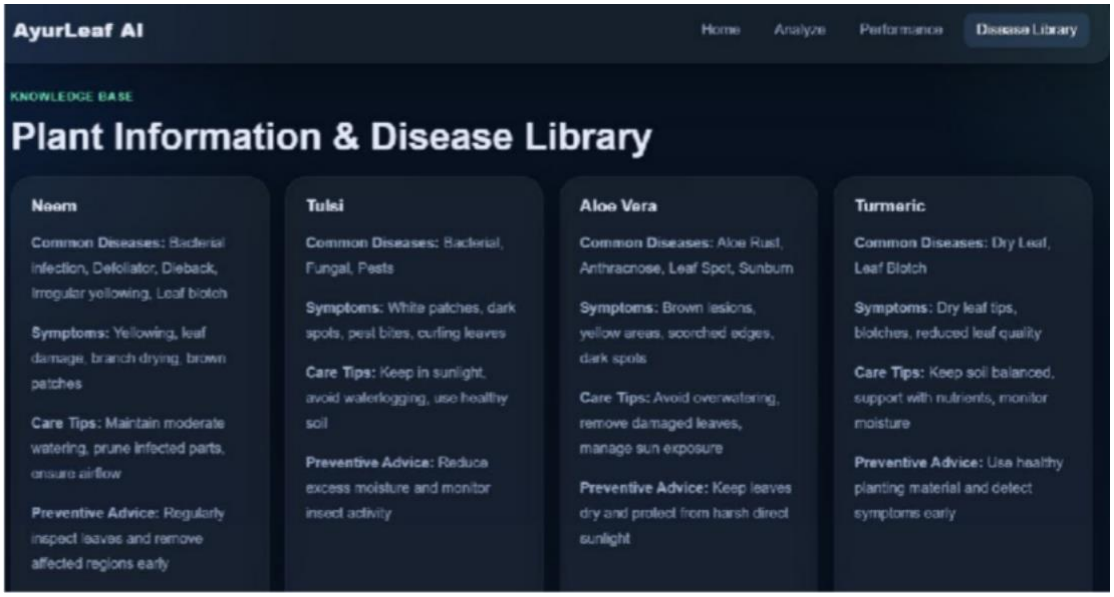

Figure 6.12: Plant and Disease Library

Chapter 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

Through this project, a full end-to-end intelligent diagnostic pipeline has been developed to detect and characterise the disease in four ayurvedic medicinal plants, namely Neem, Tulsi, Turmeric and Aloe Vera, using the deep learning and computer vision. The major efforts of this paper can be summarized by the following. An instance of a transfer learning classifier that is based on MobileNet V2 was trained on a dataset consisting of 13460 images of leaves and comprised of 18 disease and healthy categories and obtained a test performance accuracy of 90.13 with a weighted F1 score of 89.87. The decision to use MobileNetV2 with depthwise separable convolution structure is a model that has a total size of 9.25 MB, relatively small enough to be integrated on mobile and edges, and deployed in the field.

CLAHE image enhancement in the LAB colour space was included as a pre-processing measure, and the colour-block is enhanced in the number of leaf images, where the disease signatures of the chromatic colour are preserved and the visibility of the disease in the input leaf images is increased without distorting the chromatic signatures of the disease recovered by the classifier. Its improvement was towards high confidence correct prediction as is evidenced by the 98.33 percent confidence of the test case on turmeric dry leaf.

Combinational HSV-based lesion segmentation using morphological post-processing can better localise the disease using the pixel, with morphological post-processing, and quantify the severity of the disease as Low, Medium and High severity depending on the percentage of the leaf area that shows disease-wise colour coding patterns. This can be used to obtain more than just class identification, but evidence of action based severity-graded treatment suggestions of the system.

Grad-CAM visualisation gives spatial explainability of model predictions, which create heatmaps that demonstrate that the model attention is towards the biologically relevant disease lesion regions of the leaf. This openness is critical in creating user trust as well as facilitating agronomic validation of automated investigations.

Environmental analysis module, contextualises the diagnosis with condition data of the field in form of temperature, humidity and soil moisture giving the agronomic remarks that enable users to interpret disease risk based on already existing environmental parameters. The leaf colour statistics module is an analysis that measures the state of the leaf physiologically by the colour and the quality of the greenness of the leaf.

All the output of any diagnostic is merged into a tabular report that could be exported in Excel format which would have plant and severity specific treatment recommendations, and this report will be easily usable by the farmers, the field extension workers and the ayurvedic practitioners.

The analysis of the confidence shows the confidence of correct predictions to be 93.4% and incorrect predictions to be 55.2% which shows that the model can be relatively reliable in indicating itself to be uncertain which is an important feature of a safe deployment of a decision-support tool. The interpretation of image by Grad-CAM visualisations justifies diagnostic interpretability of the system, as model attention is based on the regions of disease lesions and not on background noise.

Overall, this project shows that a combined deep learning pipeline consisting of transfer learning, image quality and lesion localisation, explainability and environmental analysis can reach clinically useful levels of performance and can be further improved and implemented in the field.

7.2 LIMITATIONS

In spite of positive outcomes, a number of restrictions of the current system should be identified. First, the dataset has a moderate level of imbalance between different classes as the number of images in each class varies between 199 and 1,559. The classes of two turmeric diseases that have less than 210 samples experience the lowest F1 per class (0.79 and 0.80), implying that it is possible to improve performance by specific sets of data collection or data incident methods including generation of synthetic images via generative adaptive networks.

Second, the environmental analysis component takes scalar values that are manually entered into the analysis component with regard to the temperature, humidity, and soil moisture. Integration with the IoT sensors or 4 weather APIs in a real-world deployment would be required, to auto populate such parameters, and this will decrease manually created and or pre-entry inputs and chances of making a mistake.

Third, the lesion parting module is based on constant HSV thresholds which are configured to the brownish-yellow range of colour of the diseases in the dataset used. Currently these thresholds or the learned segmentation model might need to be recalibrated or substituted with a new segmentation model depending on the disease conditions or the colour of the lesions in plants of different species.

Fourth, the system leaf image process is one image per time. Multiple leaves of one plant or field inspection image cannot be batched processed to support large throughput in large-scale agronomic monitoring situations.

Fifth, the system is yet to be tested in a real field. All the performance measures reported in this project were achieved on laboratory-split set that was generated on the same distribution as used in the training data. Images obtained in the field, having natural background, different sources of light and partial blockage could provide new issues which could be evaluated separately.

7.3 FUTURE WORK

Some of the probable directions that can be used to expand this work are noted. Expansion of the dataset would be the most effective short-term intervention: firstly, the underserved classes of the turmeric disease would be expanded, and secondly, it would cover more economically valuable but not studied in the existing disease detection environments Ashwagandha (Withania somnifera), Brahmi (Bacopa monnieri), and Giloy (Tinospora cordifolia).

The discriminative performance of the MobileNetV2 base model can probably be enhanced by fine-tuning the upper layers instead of freezing the entire base in terms of preventing a more favorable result of the discriminative process, especially those of

visually related intra-species disease examples. A slow unfreezing method, which phases the unfreezing and training of deeper layers at progressively steeper learning rates has been found to give uniform improvements in accuracy without overfitting.

The rule-based HSV lesion segmentation could be substituted by specific semantic lesion segmentation model like U-Net that would offer more powerful and precise pixel-level disease localisation in wide range of disease conditions and in different light conditions. U-Net trained on lesion mask annotations have shown good leaf segmentation tasks in literature.

An addition of a live IoT sensor interface to retrieve the temperature, humidity and soil moisture levels in real time would convert the existing manually-driven environmental analysis module to a fully automated sub-component allowing it to be completely deployed in a smart farming setup.

It could be enhanced by developing a mobile application interface, which would utilize the small size of the MobileNetV2 model to make inferences on the device, not relying on the connection to a cloud server, which will play a crucial role when used in rural environments with unreliable internet connectivity. The TensorFlow Lite frameworks are very appropriate to this extent.

Lastly, temporal disease tracking: tracking disease evolution in multiple images of the same plant, time-stamped, would be of great diagnostic value, allowing to alert at the increasing extent of the disease severity, the ratification of Low to High levels of the disease, and to understand the effect of the treatment with time.

REFERENCES

[1] A. Mohanadevi, R. Sathya, V. C. Bharathi, S. Ananthi, G. Vijaya and K. S. Manojee, "Enhanced Medicinal Plants Identification and Classification Using CNN and MobileNetV2," *2026 5th International Conference on Communication, Computing and Electronics Systems (ICCCES)*, Coimbatore, India, 2026, pp. 1013-1017, doi: 10.1109/ICCCES62661.2026.11437263.

[2] V. Tanwar, V. Anand, R. Chauhan, R. S. Rawat and G. R. Kumar, "Enhanced Classification of Turmeric Leaf Disease Severity Levels Using an Optimized Hybrid Deep Learning Model: LevelPrecisionRecallF1-scoreAccuracy," *2024 International Conference on E-mobility, Power Control and Smart Systems (ICEMPS)*, Thiruvananthapuram, India, 2024, pp. 1-5, doi: 10.1109/ICEMPS60684.2024.10559345.

[3] S. V. K and R. S. Kumar, "A Comprehensive Analysis on Severity Estimation of Plant Diseases by Deep Learning Models," *2025 International Conference on Computational, Communication and Information Technology (ICCCIT)*, Indore, India, 2025, pp. 240-245, doi: 10.1109/ICCCIT62592.2025.10927900.

[4] A. K, H. K and K. S, "AgriVision: Deep Learning-Based Plant Disease with Daily Progress Tracking and Severity Classification," *2026 4th International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, Ottapalam, India, 2026, pp. 1622-1627, doi: 10.1109/IDCIoT67589.2026.11455647.

[5] A. Iqbal, "PlantHealthNet: Transformer-Enhanced Hybrid Models for Disease Diagnosis and Severity Estimation in Agriculture," in *IEEE Access*, vol. 13, pp. 101160-101176, 2025, doi: 10.1109/ACCESS.2025.3576990

[6] J. Lande, P. Nasra, S. Bhatt and A. Joshi, "Transfer Learning-Based Plant Leaf Disease Classification Using MobileNetV2 on the Egyptian Plant Leaf Image Dataset," *2025 4th International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*, Salem, India, 2025, pp. 813-818, doi: 10.1109/ICAAIC64647.2025.11331035.

[7] A. R. Revathi, N. B, S. M. Vinod and N. J. Warrier, "An Efficient Transfer Learning Approach for Identification and Severity Prediction of Plant Diseases," *2025 11th International Conference on Communication and Signal Processing (ICCSP)*, Melmaruvathur, India, 2025, pp. 572-577, doi: 10.1109/ICCSP64183.2025.11088680.

[8] H. R and V. K. N, "Insights on Assessing Image Processing Approaches Towards Health Status of Plant Leaf," *2022 Third International Conference on Smart*

Technologies in Computing, Electrical and Electronics (ICSTCEE), Bengaluru, India, 2022, pp. 1-7, doi: 10.1109/ICSTCEE56972.2022.10099507.

[9] R. Kumar, A. Chug and A. P. Singh, "Plant Leaf Diseases Severity Estimation using Fine-Tuned CNN Models," *2023 6th International Conference on Information Systems and Computer Networks (ISCON)*, Mathura, India, 2023, pp. 1-6, doi: 10.1109/ISCON57294.2023.10111948.

[10] L. V. Pillai and R. K. Megalingam, "FPGA-Based Plant Health Monitoring System Using Image Processing and Automated Alarm Triggering," *2025 8th International Conference on Circuit, Power & Computing Technologies (ICCPCT)*, Kollam, India, 2025, pp. 423-428, doi: 10.1109/ICCPCT65132.2025.11176762.

[11] T. Devi., K. Alice and N. Deepa, "Plant Disease Detection Using Enhanced EfficientNet Architecture in Comparison with DenseNet to Analyze the Severity in Leaves with Performance Measures," *2022 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI)*, Chennai, India, 2022, pp. 1-5, doi: 10.1109/ICDSAAI55433.2022.10028850.

[12] P. Kambli, V. Satrasala, S. K, Shreyash, S. Sharma and Y. N. Neralkar, "Identification of Ayurvedic Plants Using Deep Learning," *2024 International Conference on Computing, Semiconductor, Mechatronics, Intelligent Systems and Communications (COSMIC)*, Mangalore, India, 2024, pp. 72-76, doi: 10.1109/COSMIC63293.2024.10871655.

[13] S. D. Babu, D. Saritha, M. Sreenivasulu, C. R. Babu, J. S. Babu and M. S. Kumar, "Identification of Different Medicinal Plants through Image Processing Using CNN-GMM," *2025 6th International Conference on Recent Advances in Information Technology (RAIT)*, Dhanbad, India, 2025, pp. 1-7, doi: 10.1109/RAIT65068.2025.11089065.

[14] K. Shivaprasad and A. Wadhawan, "Deep Learning-based Plant Leaf Disease Detection," *2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, Madurai, India, 2023, pp. 360-365, doi: 10.1109/ICICCS56967.2023.10142857.

[15] R. Selvaraj, M. S. G. Devasena, T. Satheesh, K. Karpagavadivu and F. Ravindaran, "Turmeric leaf disease detection using optimized deep learning model," *2nd International Conference on Computer Vision and Internet of Things (ICCVIoT 2024)*, Coimbatore, India, 2024, pp. 248-253, doi: 10.1049/icp.2024.4431.

[16] S. S. Patil *et al.*, "Tulsi Leaf Disease Detection using CNN," *2022 IEEE Conference on Interdisciplinary Approaches in Technology and Management for*

Social Innovation (IATMSI), Gwalior, India, 2022, pp. 1-4, doi: 10.1109/IATMSI56455.2022.10119405.

[17] J. B. Guptha and E. Elakiya, "Detection of Neem Leaf Disease Using Deep Learning Models," *2025 International Conference on Data Science and Business Systems (ICDSBS)*, Chennai, India, 2025, pp. 1-7, doi: 10.1109/ICDSBS63635.2025.11031880.

[18] K. Saini and P. Agarwal, "Aloe Vera Disease Prediction using Machine Learning Techniques," *2024 5th International Conference on Smart Electronics and Communication (ICOSEC)*, Trichy, India, 2024, pp. 1117-1122, doi: 10.1109/ICOSEC61587.2024.10722720.

[19] P. Bhowmik *et al*., "Machine Learning Models can help Early Disease Detection in Medicinal Plants," *2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, Faridabad, India, 2023, pp. 106-112, doi: 10.1109/ICAICCIT60255.2023.10465856.

[20] W. Sue-Chayapak, W. Suhren and A. Srikaew, "A Novel Multi-Task Deep Learning for Plant Classification and Disease Diagnosis from Leaf Images," *2024 International Conference on Power, Energy and Innovations (ICPEI)*, Nakhon Ratchasima, Thailand, 2024, pp. 109-113, doi: 10.1109/ICPEI61831.2024.10749297.

[21] P. G. K and S. T. J, "Deep Learning-Based Identification and Classification of Ayurvedic Medicinal Plants," *2025 5th International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, MANDYA, India, 2025, pp. 1-6, doi: 10.1109/ICERECT65215.2025.11377375

[22] G. Singh and D. s. Chabra, "Plant Disease Detection using Convolution Neural Network Approach," *2023 International Conference on Inventive Computation Technologies (ICICT)*, Lalitpur, Nepal, 2023, pp. 352-355, doi: 10.1109/ICICT57646.2023.10134217.

[23] D. Yaswanth, S. Sai Manoj, M. Srikanth Yadav and E. Deepak Chowdary, "Plant Leaf Disease Detection Using Transfer Learning Approach," *2024 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, Bhopal, India, 2024, pp. 1-6, doi: 10.1109/SCEECS61402.2024.10482053.

[24] P. Vemulakonda, T. Rachamalla, R. Vaddi and S. Afreen, "A Deep Learning Based Plant Analysis for Ayurvedic Applications," *2024 5th International Conference on Innovative Trends in Information Technology (ICITIIT)*, Kottayam, India, 2024, pp. 1-6, doi: 10.1109/ICITIIT61487.2024.10580568.

APPENDIX

APPENDIX A: DATASET CLASS LABELS

The following 18 class labels constitute the classification targets of the trained model, following the naming convention PlantName__ConditionName as established during dataset flattening:

Class Index	Class Label
0	Aloe vera___Aloe_Rust
1	Aloe vera___Anthracnose
2	Aloe vera___Healthy
3	Aloe vera___Leaf_Spot
4	Aloe vera___Sunburn
5	Neem___Bacterial_infection
6	Neem___Defoliator
7	Neem___Dieback
8	Neem___Healthy
9	Neem___Irregular_yellowing
10	Neem___Leaf_blotch
11	tulsi___Bacterial
12	tulsi___Fungal
13	tulsi___Healthy
14	tulsi___Pests
15	turmeric___Dry_Leaf
16	turmeric___Healthy
17	turmeric___Leaf_Blotch

APPENDIX B: SNIPPET CODES

```
import os
import cv2
import shutil
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import splitfolders
import tensorflow as tf

from google.colab import drive, files
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Input, GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
if os.path.exists(split_dir):
    shutil.rmtree(split_dir)

splitfolders.ratio(flat_dir, output=split_dir, seed=42, ratio=(0.7, 0.2, 0.1))

print("Split folders:", os.listdir(split_dir))
print("Train classes:", os.listdir(os.path.join(split_dir, 'train')))
print("Val classes:", os.listdir(os.path.join(split_dir, 'val')))
print("Test classes:", os.listdir(os.path.join(split_dir, 'test')))
```

```
def make_gradcam_heatmap(img_array, model, last_conv_layer_name):
    grad_model = Model(
        [model.inputs],
        [model.get_layer(last_conv_layer_name).output, model.output]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        pred_index = tf.argmax(predictions[0])
        class_channel = predictions[:, pred_index]

    grads = tape.gradient(class_channel, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

    conv_outputs = conv_outputs[0]
    heatmap = tf.reduce_sum(conv_outputs * pooled_grads, axis=-1)

    heatmap = tf.maximum(heatmap, 0) / (tf.reduce_max(heatmap) + 1e-8)
    return heatmap.numpy()
```

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

acc = accuracy_score(y_true, y_pred)
prec_weighted = precision_score(y_true, y_pred, average='weighted')
rec_weighted = recall_score(y_true, y_pred, average='weighted')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

prec_macro = precision_score(y_true, y_pred, average='macro')
rec_macro = recall_score(y_true, y_pred, average='macro')
f1_macro = f1_score(y_true, y_pred, average='macro')

print("========== MODEL PERFORMANCE ==========")
print(f"Accuracy            : {acc:.4f}")
print(f"Weighted Precision  : {prec_weighted:.4f}")
print(f"Weighted Recall     : {rec_weighted:.4f}")
print(f"Weighted F1 Score   : {f1_weighted:.4f}")
print(f"Macro Precision     : {prec_macro:.4f}")
print(f"Macro Recall        : {rec_macro:.4f}")
print(f"Macro F1 Score      : {f1_macro:.4f}")
print("=======================================")
```

```
========== MODEL PERFORMANCE ==========
Accuracy            : 0.9013
Weighted Precision  : 0.9034
Weighted Recall     : 0.9013
Weighted F1 Score   : 0.8987
Macro Precision     : 0.8645
Macro Recall        : 0.8468
Macro F1 Score      : 0.8461
=======================================
```

```python
import numpy as np
import matplotlib.pyplot as plt

# confidence scores from model output
y_conf = np.max(pred, axis=1)

# separate correct and incorrect predictions
correct_mask = (y_pred == y_true)
correct_conf = y_conf[correct_mask]
incorrect_conf = y_conf[~correct_mask]

# boxplot comparison
plt.figure(figsize=(8, 5))
plt.boxplot([correct_conf, incorrect_conf], labels=["Correct Predictions", "Incorrect Predictions"])
plt.title("MobileNetV2 Confidence Comparison")
plt.ylabel("Confidence")
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

print("Mean confidence (Correct Predictions)  :", round(float(np.mean(correct_conf)), 4))
print("Mean confidence (Incorrect Predictions):", round(float(np.mean(incorrect_conf)), 4))
print("Max confidence (Correct Predictions)   :", round(float(np.max(correct_conf)), 4))
print("Max confidence (Incorrect Predictions) :", round(float(np.max(incorrect_conf)), 4))
print("Min confidence (Correct Predictions)   :", round(float(np.min(correct_conf)), 4))
print("Min confidence (Incorrect Predictions) :", round(float(np.min(incorrect_conf)), 4))
```

```
bg_mask, bg_removed = remove_background(enhanced)
lesion_mask, lesion = segment_lesion(bg_removed)
```

```
def remove_background(enhanced_img):
    gray = cv2.cvtColor(enhanced_img, cv2.COLOR_RGB2GRAY)

    # threshold for leaf vs background
    _, bg_mask = cv2.threshold(gray, 25, 255, cv2.THRESH_BINARY)

    # clean mask
    kernel = np.ones((3, 3), np.uint8)
    bg_mask = cv2.morphologyEx(bg_mask, cv2.MORPH_OPEN, kernel)
    bg_mask = cv2.morphologyEx(bg_mask, cv2.MORPH_CLOSE, kernel)

    bg_removed = cv2.bitwise_and(enhanced_img, enhanced_img, mask=bg_mask)
    return bg_mask, bg_removed


def segment_lesion(bg_removed_img):
    hsv = cv2.cvtColor(bg_removed_img, cv2.COLOR_RGB2HSV)

    # tune these if needed for your leaf images
    lower = np.array([10, 30, 30])
    upper = np.array([35, 255, 255])

    lesion_mask = cv2.inRange(hsv, lower, upper)

    kernel = np.ones((3, 3), np.uint8)
    lesion_mask = cv2.morphologyEx(lesion_mask, cv2.MORPH_OPEN, kernel)
    lesion_mask = cv2.morphologyEx(lesion_mask, cv2.MORPH_CLOSE, kernel)

    lesion = cv2.bitwise_and(bg_removed_img, bg_removed_img, mask=lesion_mask)
    return lesion_mask, lesion
```

```
-import pandas as pd

final_report_data = {
    "Predicted Class": [predicted_class],
    "Confidence": [round(float(confidence), 4)],
    "Leaf Health Status": [leaf_health_status],
    "Leaf Color Quality": [color_quality],
    "Severity Level": [severity],
    "Severity Ratio": [round(float(ratio), 4)],
    "Temperature (C)": [temperature],
    "Humidity (%)": [humidity],
    "Soil Moisture (%)": [soil_moisture],
    "Image Status": [status],
    "Suggestion": [suggestion]
}

report_df = pd.DataFrame(final_report_data)

excel_path = "/content/final_plant_diagnosis_report.xlsx"
report_df.to_excel(excel_path, index=False)

print("Excel report saved at:", excel_path)
report_df
```